



# BCSE332L-Deep Learning

Dr. Rohith G

Assistant Professor Senior, CAMIT

School of Electronics Engineering (SENSE)

# Contents

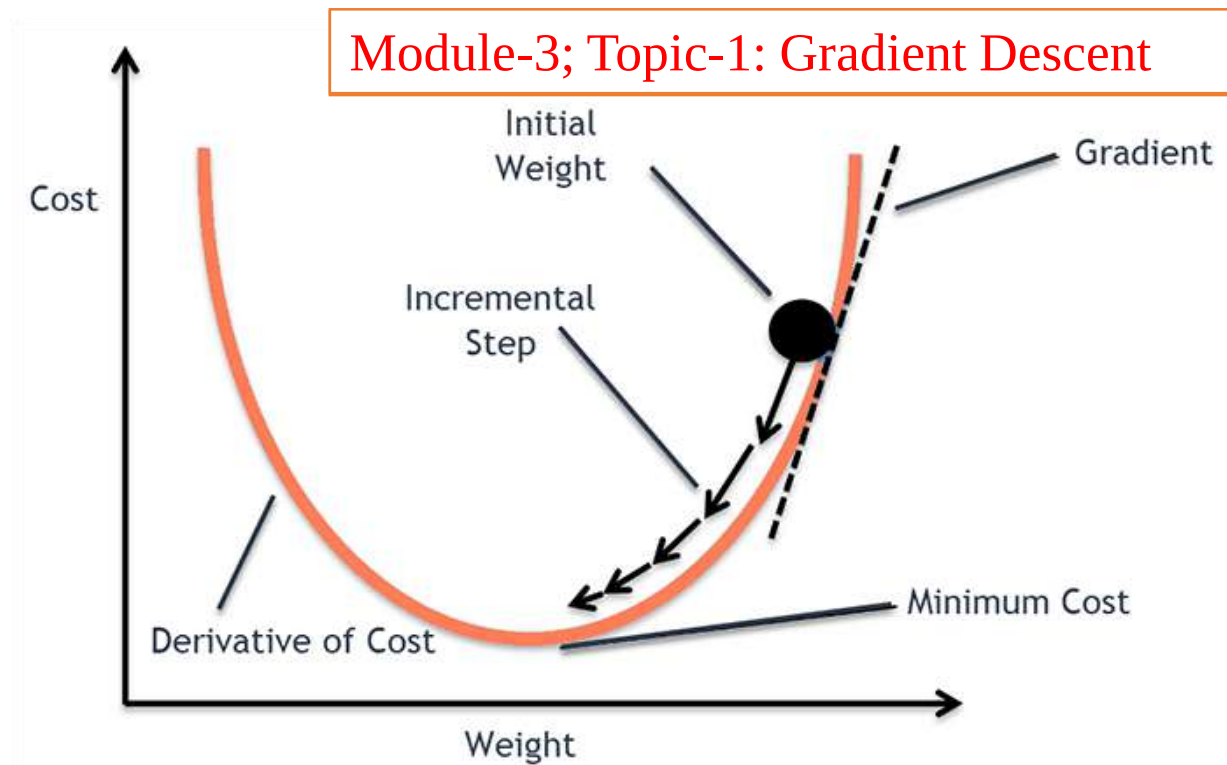
- **Module-1:** Introduction to neural networks and deep neural networks
  - **Module-2:** Convolution neural networks
  - **Module-3:** Improving deep neural networks
  - **Module-4:** Recurrent networks
  - **Module-5:** Recursive neural networks
  - **Module-6:** Advanced Neural networks
  - **Module-7:** Deep reinforcement learning
- **Mini-batch Gradient Descent – Exponential Weighted Averages-Gradient Descent with Momentum**
  - **RMSProp and Adam Optimization**
  - **Hyperparameter tuning**
  - **Batch Normalization**
  - **Softmax Regression**
  - **Softmax classifier**
  - **Deep Learning Frameworks – Data Augmentation**
  - **Under-fitting Vs Over-fitting**

# Optimizing Functions

- Gradient Descent: first-order, iterative-based optimization algorithm
- Momentum Optimizer: accelerates the stochastic gradient descent in the relevant direction
- Adagrad: adaptively scales learning rate for different dimension

## Common Variants of Gradient Descent:

- **Stochastic Gradient Descent (SGD):** Updates weights using a random subset (mini-batch) of the training data, making it faster but more noisy.
- **Mini-Batch Gradient Descent:** A compromise between Batch Gradient Descent (which uses the entire dataset for each update) and SGD, using small batches to reduce computational overhead.
- **Adam (Adaptive Moment Estimation):** Combines ideas from momentum and RMSprop, adjusting the learning rate for each parameter dynamically.
- **RMSprop:** Adapts the learning rate for each parameter by dividing the gradient by a moving average of its recent magnitude.



# Gradient descent

An algorithm to minimize a cost function by optimizing its parameters.

We start with random guess and slowly move to right or best answer.

Need - **Parameter Optimization**

**Formula:**

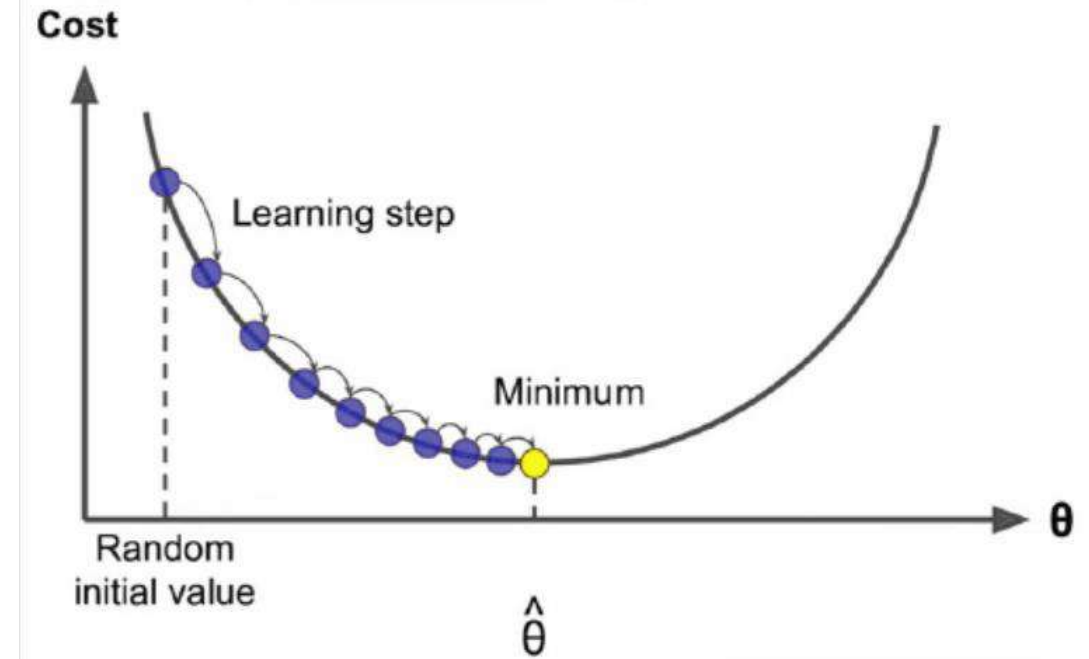
**New value = old value - step size**

Where,

**step size = Learning rate x Slope.**

**Note:**

If the learning rate is too high, you might jump across the valley and end up on the other side, possibly even higher up than you were before.



This might make the algorithm diverge, with larger values, failing to find a good solution.

# Variants of gradient descent

## Batch gradient descent:

- Vanilla gradient descent, is the simplest variant of gradient descent.
- In batch gradient descent, *the entire training dataset is used to compute the gradients of the cost function with respect to the model parameters in each iteration.*
- This can be computationally expensive for large datasets, but it guarantees convergence to a local minimum of the cost function.

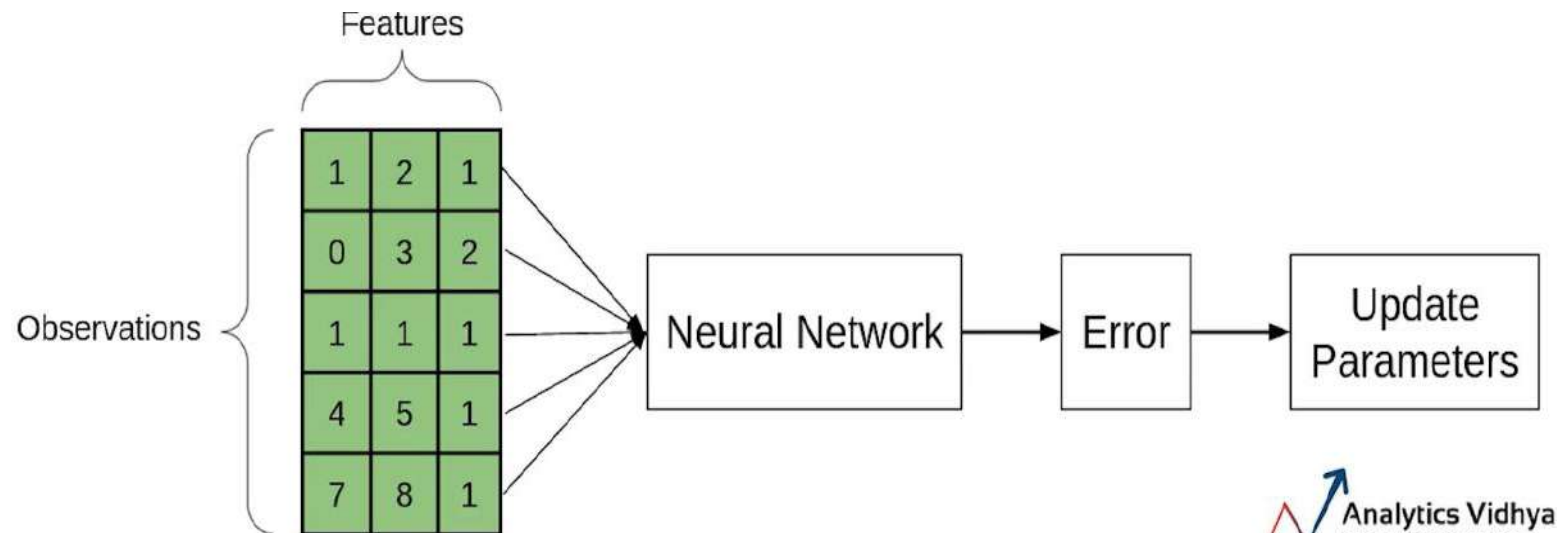
$$\theta = \theta - \alpha \cdot \nabla J(\theta)$$

where:

$\theta$  is the parameter vector

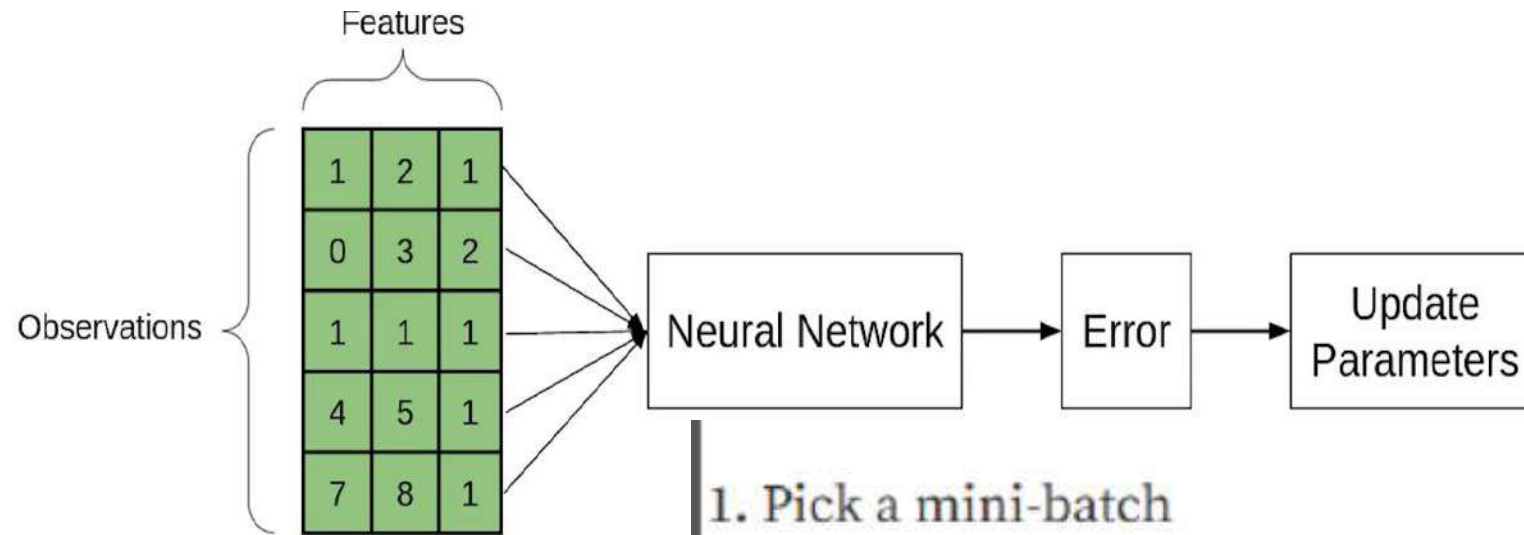
$\alpha$  is the learning rate

$\nabla J(\theta)$  is the gradient of the cost function  $J$  with respect to  $\theta$



# Variants of gradient descent

## Batch gradient descent:

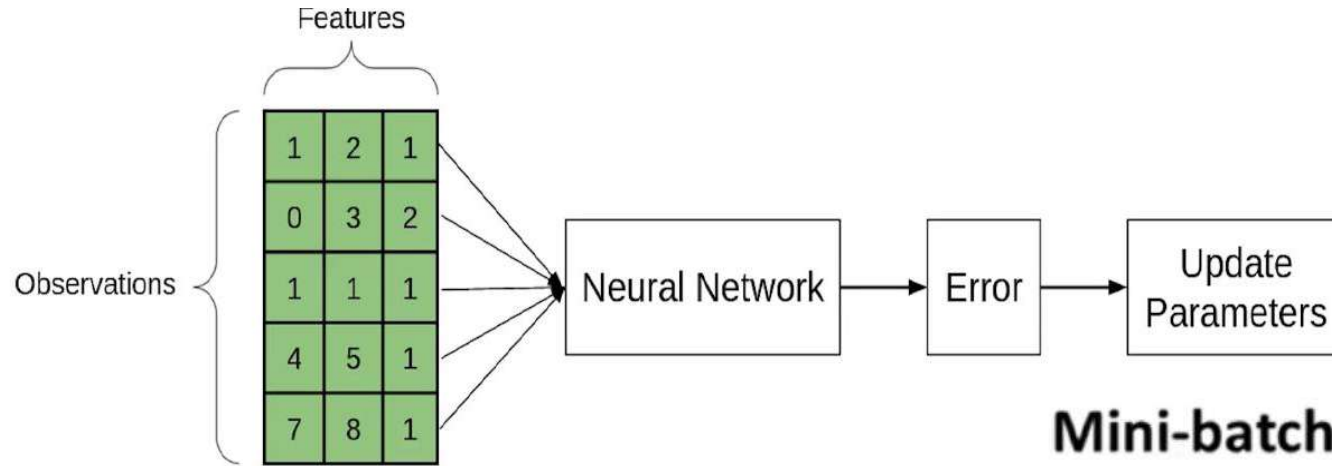


1. Pick a mini-batch
2. Feed it to Neural Network
3. Calculate the mean gradient of the mini-batch
4. Use the mean gradient we calculated in step 3 to update the weights
5. Repeat steps 1-4 for the mini-batches we created



# Variants of gradient descent

## Batch gradient descent:



## Mini-batch gradient descent

Say  $b = 10, m = 1000$ .

Repeat {

for  $i = 1, 11, 21, 31, \dots, 991$  {

$$\theta_j := \theta_j - \alpha \frac{1}{10} \sum_{k=i}^{i+9} (h_{\theta}(x^{(k)}) - y^{(k)}) x_j^{(k)}$$

(for every  $j = 0, \dots, n$ )

}

}

# Batch Gradient-An illustration

Consider a neural network with:

- **One input layer** (with 2 input features).
- **One hidden layer** with 2 neurons.
- **One output layer** with a single output neuron.

The objective is to minimize this loss using **Gradient Descent**.

## 1. Hidden Layer:

- Compute  $z^{[1]}$ :

$$z^{[1]} = X \cdot W^{[1]} + b^{[1]} = \begin{bmatrix} 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$$

$$z^{[1]} = \begin{bmatrix} 1(0.1) + 2(0.3) & 1(0.2) + 2(0.4) \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$$

$$z^{[1]} = \begin{bmatrix} 0.7 & 1.0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 1.1 \end{bmatrix}$$

- Now, apply the sigmoid function  $\sigma(z)$ :

$$a^{[1]} = \sigma(z^{[1]}) = \begin{bmatrix} \sigma(0.8) \\ \sigma(1.1) \end{bmatrix}$$

Using  $\sigma(x) = \frac{1}{1+e^{-x}}$ , we get:

$$\sigma(0.8) \approx 0.6899 \quad \text{and} \quad \sigma(1.1) \approx 0.7503$$

So, the activation from the hidden layer is:

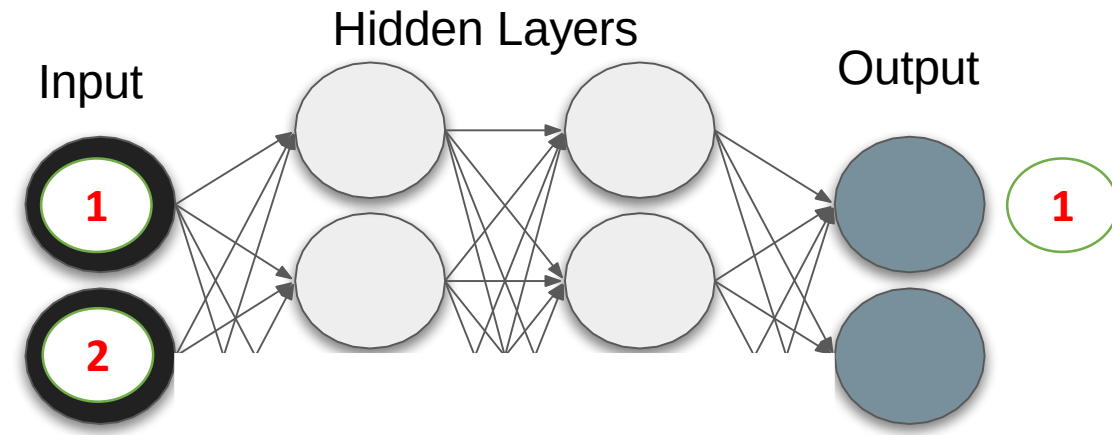
$$a^{[1]} = \begin{bmatrix} 0.6899 \\ 0.7503 \end{bmatrix}$$

## Module-3; Topic-1: Gradient Descent

- Input  $X = [1, 2]$
- Actual output  $y = 1$

Neural Network Parameters:

- Weights for the hidden layer  $W^{[1]} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$
- Bias for the hidden layer  $b^{[1]} = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$
- Weights for the output layer  $W^{[2]} = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$
- Bias for the output layer  $b^{[2]} = 0.2$





# Batch Gradient-An illustration

## 2. Output Layer:

- Compute  $z^{[2]}$ :

$$z^{[2]} = a^{[1]} \cdot W^{[2]} + b^{[2]} = \begin{bmatrix} 0.6899 & 0.7503 \end{bmatrix} \cdot \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix} + 0.2$$

$$z^{[2]} = 0.6899(0.5) + 0.7503(0.6) + 0.2 = 0.34495 + 0.45018 + 0.2 = 0.99513$$

- Apply the sigmoid activation:

$$a^{[2]} = \sigma(0.99513) \approx 0.7305$$

## Step 3: Calculate the Loss (Cost Function)

We'll use mean squared error (MSE) for the loss function. The formula is:

$$\text{Loss} = \frac{1}{2}(y - a^{[2]})^2$$

Where:

- $y = 1$  is the actual output.
- $a^{[2]} = 0.7305$  is the predicted output.

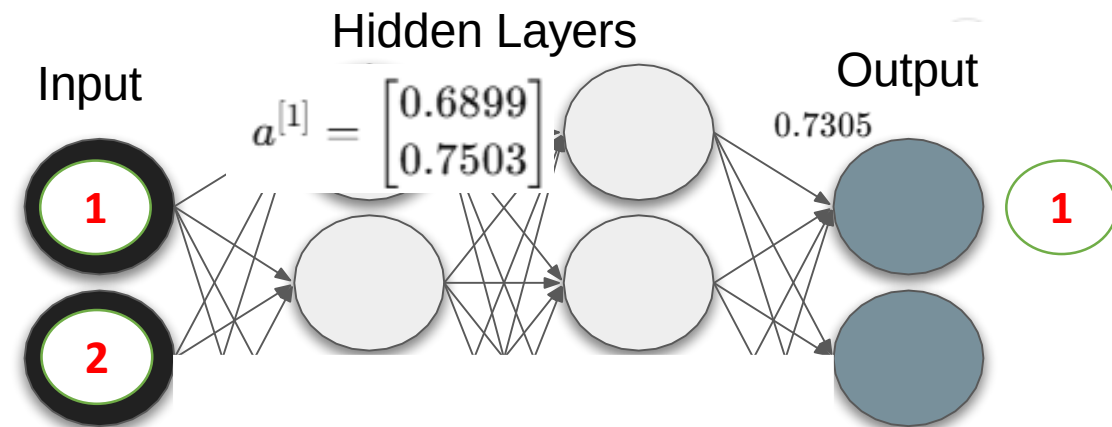
$$\text{Loss} = \frac{1}{2}(1 - 0.7305)^2 = \frac{1}{2}(0.2695)^2 = 0.0363$$

## Module-3; Topic-1: Gradient Descent

- Input  $X = [1, 2]$
- Actual output  $y = 1$

Neural Network Parameters:

- Weights for the hidden layer  $W^{[1]} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$
- Bias for the hidden layer  $b^{[1]} = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$
- Weights for the output layer  $W^{[2]} = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$
- Bias for the output layer  $b^{[2]} = 0.2$



# Batch Gradient-An illustration

## 1. Output Layer Gradient:

- The derivative of the loss with respect to the output  $z^{[2]}$  is:

$$\frac{\partial \text{Loss}}{\partial z^{[2]}} = (a^{[2]} - y) \cdot \sigma'(z^{[2]})$$

Where  $\sigma'(z^{[2]}) = a^{[2]}(1 - a^{[2]})$ .

So, we get:

$$\frac{\partial \text{Loss}}{\partial z^{[2]}} = (0.7305 - 1) \cdot 0.7305 \cdot (1 - 0.7305)$$

$$\frac{\partial \text{Loss}}{\partial z^{[2]}} = -0.2695 \cdot 0.7305 \cdot 0.2695 \approx -0.0533$$

- Next, we compute the gradient with respect to the weights and bias of the output layer:

$$\frac{\partial \text{Loss}}{\partial W^{[2]}} = a^{[1]} \cdot \frac{\partial \text{Loss}}{\partial z^{[2]}}$$

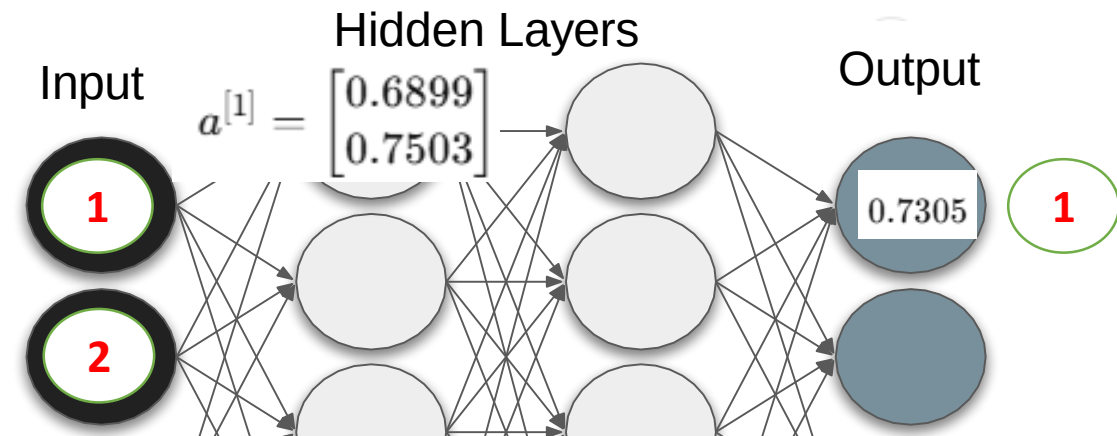
$$\frac{\partial \text{Loss}}{\partial W^{[2]}} = \begin{bmatrix} 0.6899 \\ 0.7503 \end{bmatrix} \cdot (-0.0533) = \begin{bmatrix} -0.0368 \\ -0.0400 \end{bmatrix}$$

## Module-3; Topic-1: Gradient Descent

- Input  $X = [1, 2]$
- Actual output  $y = 1$

Neural Network Parameters:

- Weights for the hidden layer  $W^{[1]} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$
- Bias for the hidden layer  $b^{[1]} = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$
- Weights for the output layer  $W^{[2]} = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$
- Bias for the output layer  $b^{[2]} = 0.2$



# Batch Gradient-An illustration

## Step 5: Gradient Descent Update

We update the parameters using the gradients computed above and the learning rate  $\eta$ . The rule is:

$$W = W - \eta \cdot \frac{\partial \text{Loss}}{\partial W}$$

Assume  $\eta = 0.1$ . The weights will be updated as follows:

- For  $W^{[2]}$ , using the gradient  $\frac{\partial \text{Loss}}{\partial W^{[2]}} = \begin{bmatrix} -0.0368 \\ -0.0400 \end{bmatrix}$ :

$$W^{[2]} = W^{[2]} - 0.1 \cdot \begin{bmatrix} -0.0368 \\ -0.0400 \end{bmatrix}$$

$$W^{[2]} = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix} - \begin{bmatrix} -0.0037 \\ -0.0040 \end{bmatrix}$$

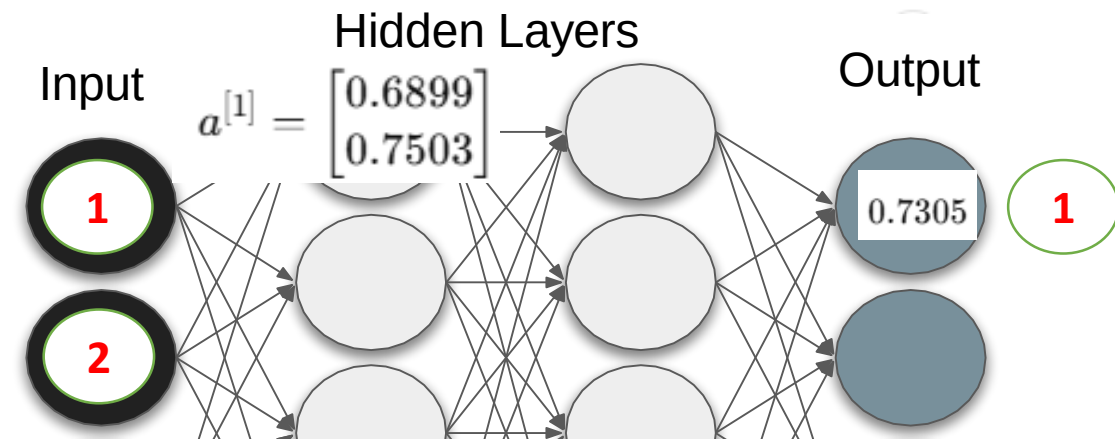
$$W^{[2]} = \begin{bmatrix} 0.5037 \\ 0.6040 \end{bmatrix}$$

This process is repeated iteratively until the loss converges to a minimum.

- Input  $X = [1, 2]$
- Actual output  $y = 1$

Neural Network Parameters:

- Weights for the hidden layer  $W^{[1]} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$
- Bias for the hidden layer  $b^{[1]} = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$
- Weights for the output layer  $W^{[2]} = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$
- Bias for the output layer  $b^{[2]} = 0.2$



# Variants of gradient descent

## 2. Mini-batch Gradient Descent

- Mini-batch gradient descent is a compromise between batch gradient descent and stochastic gradient descent.
- In mini-batch gradient descent, the gradients are computed on a small random subset of the training dataset, typically between 10 and 1000 examples, called a **mini-batch**.

$$\theta = \theta - \alpha \cdot \frac{1}{m} \sum_{i=1}^m \nabla J_i(\theta)$$

where:

$\theta$  is the parameter vector

$\alpha$  is the learning rate

$m$  is the mini-batch size

$\nabla J_i(\theta)$  is the gradient of the cost function  $J$  with respect to  $\theta$  computed on a mini-batch of size  $m$

|          |   |   |
|----------|---|---|
| Features |   |   |
| 1        | 2 | 1 |
| 0        | 3 | 2 |
| 1        | 1 | 1 |
| 4        | 5 | 1 |
| 7        | 8 | 1 |

Observations

- This reduces the computational cost of the algorithm compared to batch gradient descent, while also reducing the variance of the updates compared to SGD.
- Mini-batch gradient descent is widely used in deep learning because it strikes a good balance between convergence speed and stability.

# Exponential Weighted Averages

A common technique is to maintain an **Exponential Moving Average (EMA)** of the model's weights during training. This helps in improving the generalization ability of the model by smoothing out the updates made to the model's weights and reducing overfitting to the training data.

$$\theta_{ema,t} = \alpha \cdot \theta_t + (1 - \alpha) \cdot \theta_{ema,t-1}$$

Where:

- $\theta_{ema,t}$  is the exponentially weighted average of the weights at time  $t$ ,
- $\alpha$  is the smoothing factor (typically a value close to 1, e.g., 0.99),
- $\theta_t$  is the current model's weights at time  $t$ ,
- $\theta_{ema,t-1}$  is the EMA of the model's weights at the previous time step.

At the end of training, the EMA weights  $\theta_{ema}$  are often used for evaluation and testing, as they typically provide better generalization than the final weights  $\theta_t$ .

# Exponential Weighted Averages

## 2. Exponential Moving Average of Gradients

Another use of EWAs in DNNs is to smooth gradients during training to reduce noise and help stabilize the optimization process, especially when using **stochastic gradient descent (SGD)** or its variants. This is similar to maintaining EMA of model weights but applies to the gradients that are used to update the model parameters.

### EMA of Gradients:

During optimization, instead of using the raw gradients  $\nabla \mathcal{L}_t$  at each iteration  $t$ , we maintain an EMA of the gradients:

$$\nabla \mathcal{L}_{ema,t} = \alpha \cdot \nabla \mathcal{L}_t + (1 - \alpha) \cdot \nabla \mathcal{L}_{ema,t-1}$$

Where:

- $\nabla \mathcal{L}_t$  is the gradient of the loss function at time  $t$ ,
- $\nabla \mathcal{L}_{ema,t}$  is the EMA of the gradients at time  $t$ ,
- $\alpha$  is the smoothing factor (again, typically close to 1).

## 3. Exponential Moving Average of Losses

Some methods maintain an EMA of the loss function (or cost function) during training. This helps in stabilizing the training process by reducing the effect of transient spikes in the loss that might occur due to noisy gradients or batch-to-batch variance. This smoothed loss can also be used as a signal for early stopping or learning rate adjustment.



# Exponential Weighted Averages

## 4. Momentum in Optimizers

In optimizers like **Momentum** and **Adam**, the concept of Exponential Weighted Averages is embedded into the algorithm itself.

- **Momentum:** In momentum-based optimization, the gradient is combined with a moving average of previous gradients to determine the update direction. This is a form of weighted averaging, where the past gradients (with higher weight) are used to "accelerate" the update in the relevant direction.

The update rule for momentum is typically:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla \mathcal{L}_t$$

$$\theta_t = \theta_{t-1} - \eta \cdot v_t$$

Where:

- $v_t$  is the velocity or weighted average of gradients at time  $t$ ,
- $\beta$  is the momentum factor (like  $\alpha$  in EMA),
- $\eta$  is the learning rate,
- $\nabla \mathcal{L}_t$  is the gradient of the loss at time  $t$ .

### Benefits of Using EWAs in Neural Networks:

- **Stabilization of training:** EWAs help reduce the impact of noisy updates or erratic gradient values, leading to smoother convergence.
- **Improved generalization:** Using EMA weights can help prevent overfitting and improve the performance on the validation or test set.
- **Faster convergence:** In some cases, using momentum or smoothed gradients can accelerate the convergence process.

# Variants of gradient descent

## 3. Stochastic Gradient Descent (SGD):

- Stochastic gradient descent is a variant of gradient descent that updates the model parameters for each training example in the dataset.
- Unlike batch gradient descent, which uses the entire dataset to compute the gradients, SGD updates the parameters based on a randomly selected training example.
- This can lead to faster convergence because the updates are more frequent and noisy, but it can also result in more oscillations in the cost function due to the randomness of the updates.

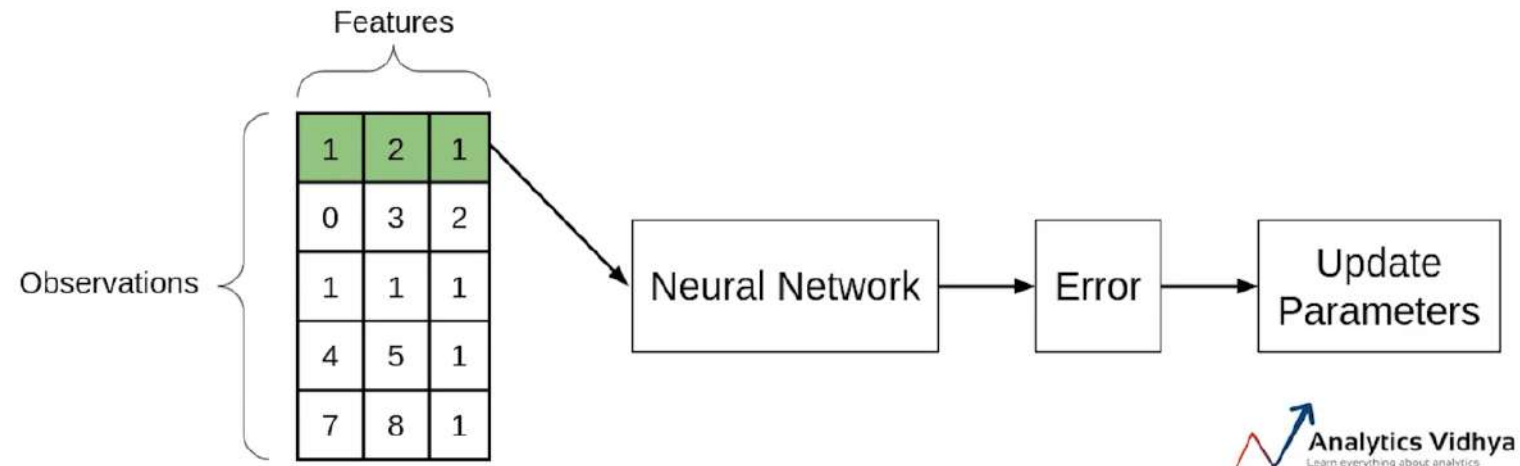
$$\theta = \theta - \alpha \cdot \nabla J_i(\theta)$$

where:

$\theta$  is the parameter vector

$\alpha$  is the learning rate

$\nabla J_i(\theta)$  is the gradient of the cost function  $J$  with respect to  $\theta$  computed on a single randomly selected training example  $i$



# Variants of gradient descent

## 4. Momentum Gradient

- Momentum gradient descent is a variant of gradient descent that adds a momentum term to the update rule.
- The momentum term accumulates the gradient values over time and dampens the oscillations in the cost function, leading to faster convergence. This is particularly useful in cases where the cost function has a lot of noise or curvature, which can cause traditional gradient descent to get stuck in local minima.

$$v_t = \gamma v_{t-1} + \alpha \cdot \nabla J(\theta_{t-1})$$

$$\theta_t = \theta_{t-1} - v_t$$

where:

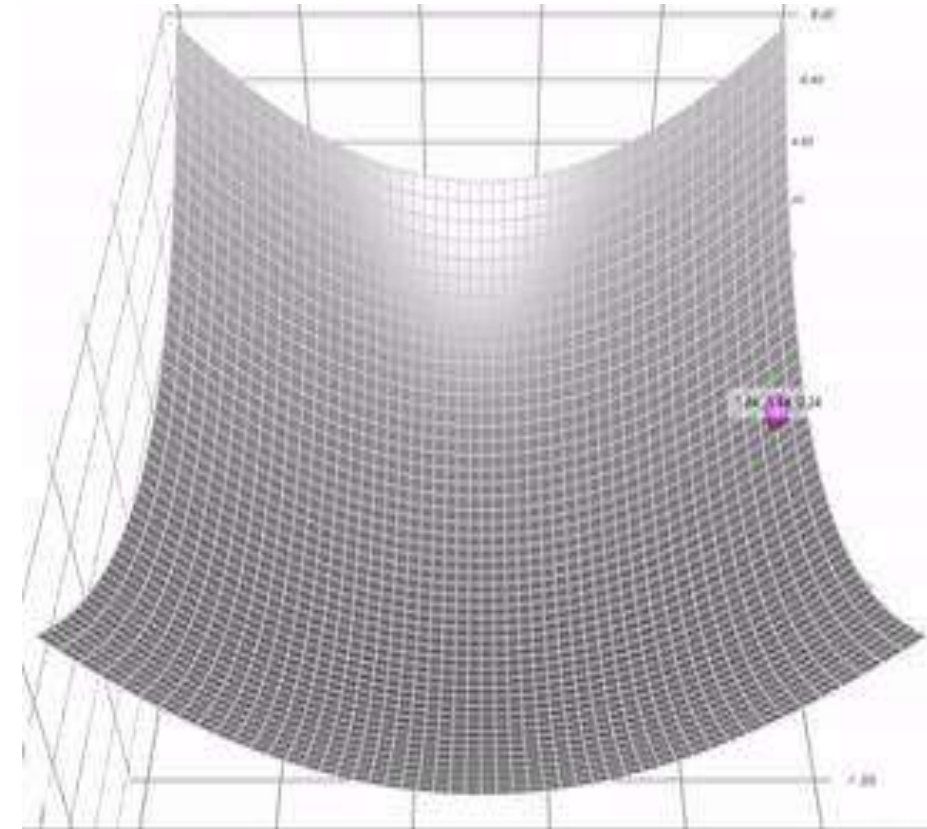
$\theta_t$  is the parameter vector at iteration  $t$

$\alpha$  is the learning rate

$\nabla J(\theta_{t-1})$  is the gradient of the cost function  $J$  with respect to  $\theta$  evaluated at  $\theta_{t-1}$

$v_t$  is the velocity vector at iteration  $t$

$\gamma$  is the momentum coefficient, which controls the contribution of the previous velocity vector to the current velocity vector





# Variants of gradient descent

## 5. Adagrad

- Adagrad is a variant of gradient descent that adapts the learning rate for each parameter based on its historical gradient values.
- Parameters with large gradients have their learning rates reduced, while parameters with small gradients have their learning rates increased. This helps to normalize the updates and can be useful in cases where the cost function has a lot of curvature or different scales of gradients.

$$g_t = \nabla J(\theta_t)$$

$$s_t = s_{t-1} + g_t^2$$

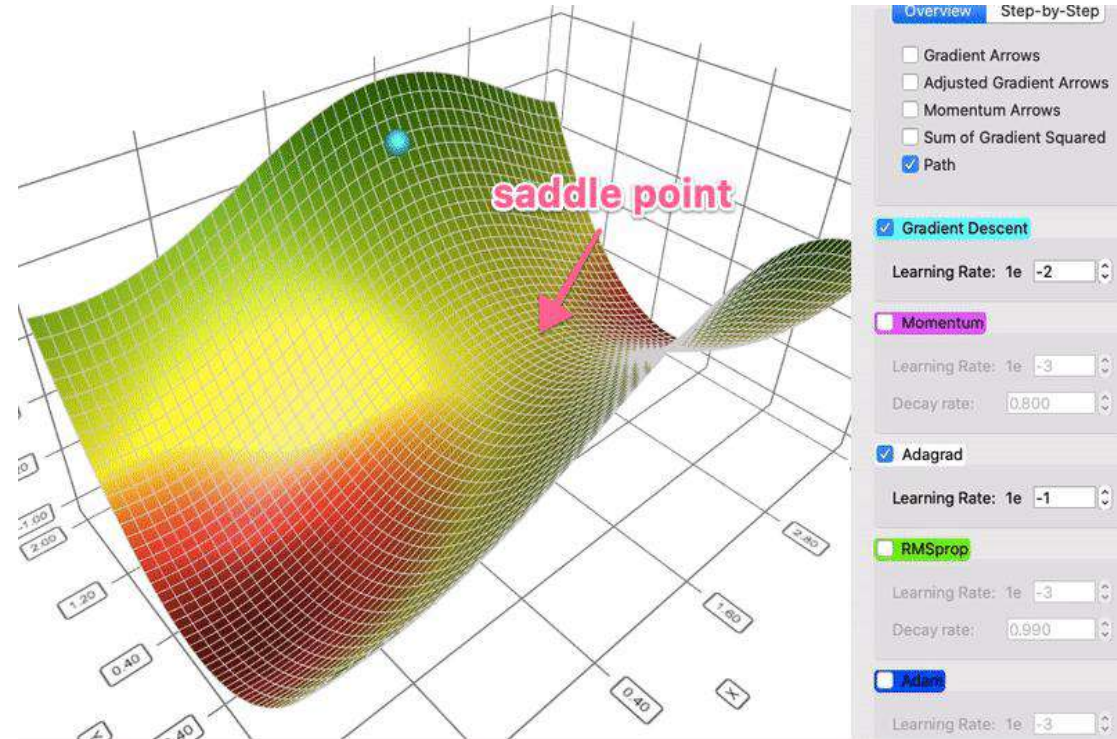
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{s_t + \epsilon}} \cdot g_t$$

where:

$\theta_t$  is the parameter vector at iteration  $t$

$\alpha$  is learning rate

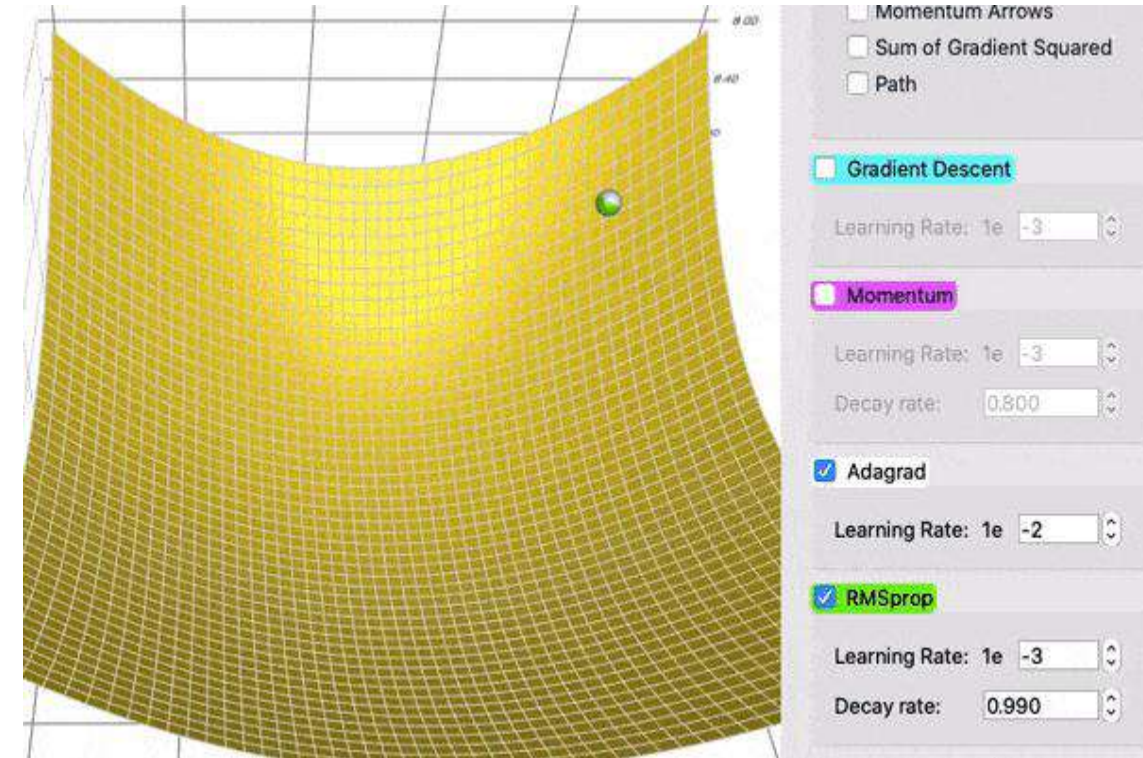
$\nabla J(\theta_t)$  is gradient of the cost function  $J$  with respect to  $\theta$  evaluated at  $\theta_t$



# Variants of gradient descent

## 6. RMSProp

- RMSProp is a variant of gradient descent that also *adapts the learning rate for each parameter, but instead of using the historical gradient values, it uses a moving average of the squared gradient values*. This helps to reduce the learning rate for parameters that have large squared gradient values, which can cause the algorithm to oscillate or diverge.



$$\text{sum\_of\_gradient\_squared} = \text{previous\_sum\_of\_gradient\_squared} * \text{decay\_rate} + \text{gradient}^2 * (1 - \text{decay\_rate})$$

$$\text{delta} = -\text{learning\_rate} * \text{gradient} / \text{sqrt}(\text{sum\_of\_gradient\_squared})$$

$$\text{theta} += \text{delta}$$

# Variants of gradient descent

## 7. Adam

- Adam, Adaptive Moment Estimation, is a variant of gradient descent that ***combines the ideas of Adagrad and RMSProp***.
- It adapts the learning rate for each parameter based on the historical gradient values and also uses a moving average of the gradient values to compute the momentum term.
- Adam is one of the most widely used optimization algorithms in deep learning because it is efficient, stable, and robust to different types of cost functions and datasets.

$$\text{sum\_of\_gradient} = \text{previous\_sum\_of\_gradient} * \text{beta1} + \text{gradient} * (1 - \text{beta1})$$

[Momentum]

$$\text{sum\_of\_gradient\_squared} = \text{previous\_sum\_of\_gradient\_squared} * \text{beta2} + \text{gradient}^2 * (1 - \text{beta2})$$

[RMSProp]

$$\text{delta} = -\text{learning\_rate} * \text{sum\_of\_gradient} / \sqrt{\text{sum\_of\_gradient\_squared}}$$

$$\text{theta} += \text{delta}$$

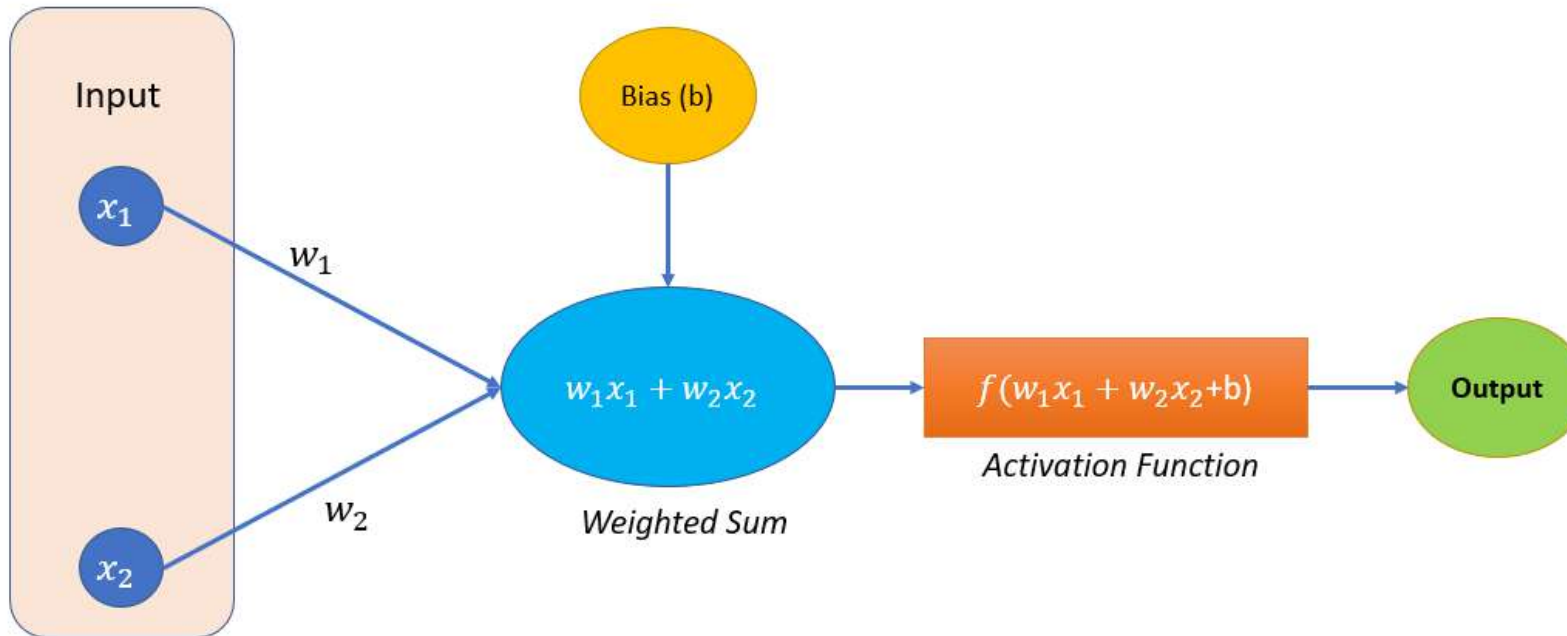


# Variants of Gradient descent optimization algorithms

| Variant                            | Advantages                                                                      | Disadvantages                                                            |
|------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Batch gradient descent</b>      | Guaranteed convergence to global optimum                                        | Computationally expensive for large datasets, slow convergence           |
| <b>Stochastic gradient descent</b> | Faster convergence, more efficient for large datasets                           | High variance, may not converge to global optimum                        |
| <b>Mini-batch gradient descent</b> | Balanced convergence speed and computational cost, efficient for large datasets | Choice of mini-batch size can be a challenge                             |
| <b>Momentum gradient descent</b>   | Faster convergence, less likely to get stuck in local minima                    | May overshoot and oscillate around the optimum                           |
| <b>Adagrad</b>                     | Adaptive learning rate, efficient for sparse data                               | Can stop learning too early                                              |
| <b>RMSProp</b>                     | Adaptive learning rate, efficient for non-stationary problems                   | Can stop learning too early, requires tuning of hyperparameters          |
| <b>Adam</b>                        | Adaptive learning rate, efficient for large datasets and noisy data             | Can converge to suboptimal solutions, requires tuning of hyperparameters |

# Parameters

**Parameters** are the internal variables of a model that are learned during training. In the context of neural networks, parameters are the values that define the model's structure and behavior. These parameters are adjusted during the training process in order to minimize the loss and improve the model's predictions.



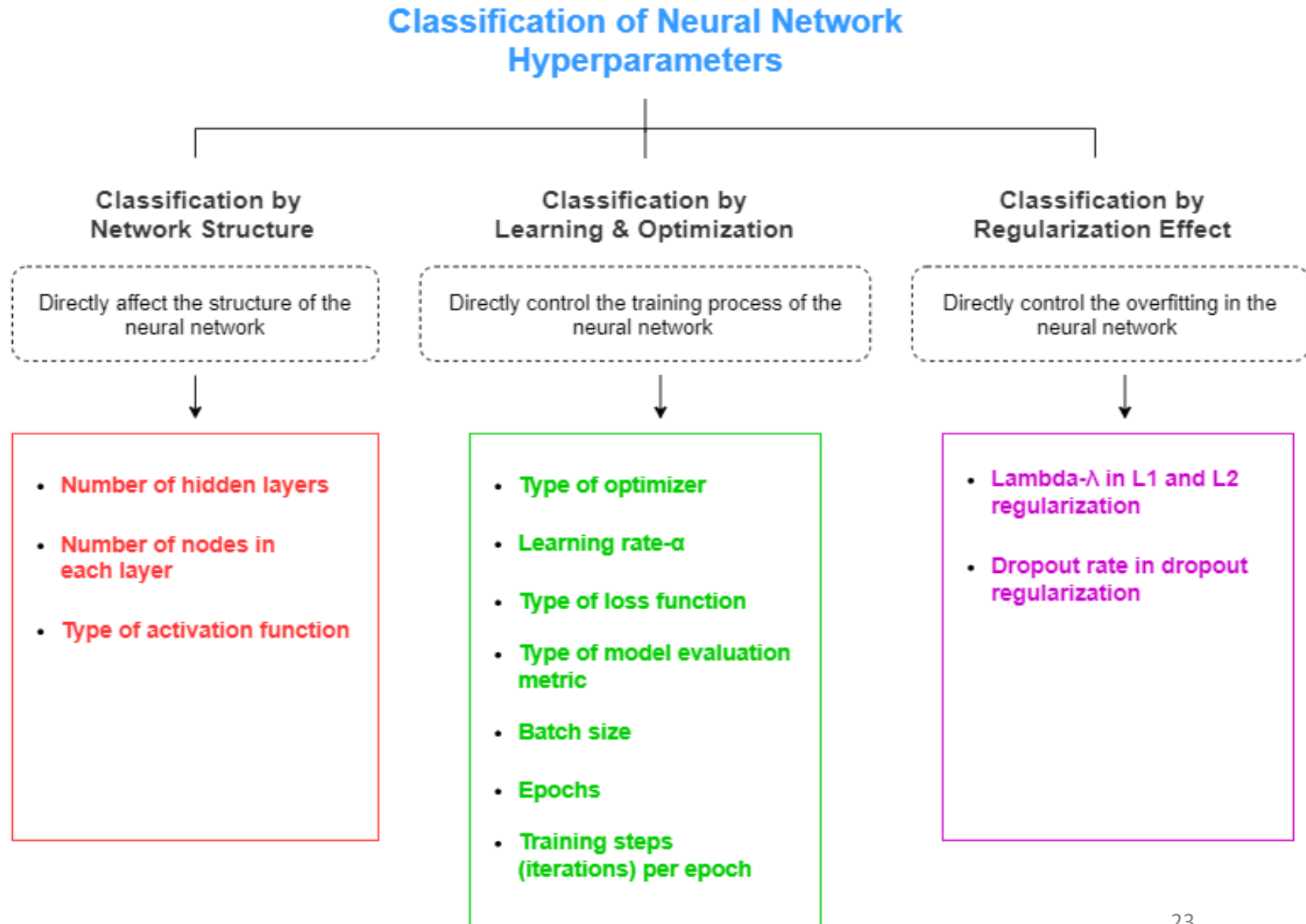
During training, the **parameters (weights and biases)** are adjusted through **backpropagation** and **gradient descent** in order to minimize the loss function.

## Examples of parameters in neural networks:

- 1. Weights (W):** These are the values that connect the neurons across layers. Each connection between two neurons has a weight that determines the strength of the signal from one neuron to the next.
- 2. Bias (b):** These are additional parameters that are added to the weighted sum before applying the activation function. Biases help shift the activation function and allow the network to learn patterns that do not pass through the origin.  
Bias allows the network to learn a broader range of data patterns, especially when the inputs are zero or very small.

# Hyper parameters

- Hyperparameters are the external configuration settings that control the training process.
- Unlike parameters, hyperparameters are not learned from the data; instead, they are set before training and can have a significant impact on the performance of the model.
- These are often tuned manually or via automated processes (like grid search or random search).



### Common hyperparameters in neural networks:

- **Learning Rate:** This determines how big a step the optimizer takes when updating the model's weights. A higher learning rate can speed up the convergence but risks overshooting the optimal solution, while a lower learning rate ensures smoother convergence but may take longer.
- **Number of Epochs:** An epoch represents one complete cycle through the entire dataset during training. The number of epochs defines how many times the model will learn from the full training data.
- **Batch Size:** This is the number of training examples utilized in one forward and backward pass. Larger batch sizes often speed up training but might reduce the accuracy of the model. Smaller batch sizes might lead to more accurate models but increase training time.
- **Number of Hidden Layers and Neurons:** In deep neural networks, you need to specify the architecture, i.e., how many hidden layers and how many neurons each hidden layer will contain. More layers and neurons generally allow the network to model more complex functions, but they can also lead to overfitting and increased computational cost.
- **Activation Function:** The choice of activation function, such as ReLU, sigmoid, or tanh, determines how the neurons' output is transformed before passing it to the next layer. It introduces non-linearity into the model, allowing it to learn more complex patterns.

**Optimization Algorithm:** This defines how the model updates its weights. Common choices include **Stochastic Gradient Descent (SGD)**, **Adam** or **RMSprop** based on how they adjust the learning rate and update weights to minimize the loss.

- **Regularization Parameters: L1 and L2 regularization:** These techniques penalize large weights to prevent overfitting.
- **Dropout Rate:** Dropout is a regularization method where a percentage of neurons are randomly dropped out during training, forcing the network to learn more robust features.
- **Momentum:** In optimization, momentum is a technique that helps accelerate gradients vectors in the right directions, thus leading to faster converging.
- **Weight Initialization:** How weights are initially set before training starts. Examples include **Random initialization** (Assigning random values to weights), **Xavier/Glorot initialization** (Designed to keep the variance of the inputs and outputs across layers in check), **He initialization** (Designed to work well with ReLU activation functions).
- **Choosing Hyperparameters:** Hyperparameters significantly affect the model's performance, but there is no one-size-fits-all set of values. Often, hyperparameter tuning requires experimentation and can be done through:
  - **Grid Search:** Systematically testing a set of hyperparameter values.
  - **Random Search:** Randomly selecting hyperparameters from a predefined range.
  - **Bayesian Optimization:** A probabilistic model that intelligently explores the space of hyperparameters.
  - **Cross-validation:** Evaluating the performance of different hyperparameter settings on a validation set.



# Hyperparameter-Tuning\_Illustration-1

You are building a neural network for image classification (e.g., on the CIFAR-10 dataset). You want to tune the following hyperparameters:

**1.Learning rate** (`lr`): [0.001, 0.01, 0.1]

**2.Batch size** (`batch_size`): [16, 32, 64]

**3.Number of layers** (`num_layers`): [2, 3, 4]

**4.Number of neurons per layer** (`neurons`): [64, 128, 256]

You decide to use **Grid Search** to explore all possible combinations of these hyperparameters. You will train your model for a fixed number of epochs (say, 10) and evaluate it using accuracy on the validation set.

The first step is to create a grid of all possible hyperparameter combinations.

Given the following choices:

- Learning rate (`lr`): 0.001, 0.01, 0.1
- Batch size (`batch_size`): 16, 32, 64
- Number of layers (`num_layers`): 2, 3, 4
- Neurons per layer (`neurons`): 64, 128, 256

We will have a total of:

Total Combinations =  $3(\text{lr}) \times 3(\text{batch size}) \times 3(\text{num layers}) \times 3(\text{neurons}) = 81$  combinations

# Hyperparameter-Tuning\_Illustration-1

For each of these 81 combinations, you will train your model and record the accuracy on the validation set. Let's assume you run the models and obtain the following results (simplified table with sample accuracy values):

| Combination | lr    | batch_size | num_layers | neurons | Accuracy (%) |
|-------------|-------|------------|------------|---------|--------------|
| 1           | 0.001 | 16         | 2          | 64      | 70.5         |
| 2           | 0.001 | 16         | 2          | 128     | 72.1         |
| 3           | 0.001 | 16         | 2          | 256     | 71.3         |
| 4           | 0.001 | 16         | 3          | 64      | 74.0         |
| 5           | 0.001 | 16         | 3          | 128     | 75.2         |
| 6           | 0.001 | 16         | 3          | 256     | 74.5         |
| 7           | 0.001 | 16         | 4          | 64      | 73.8         |
| 8           | 0.001 | 16         | 4          | 128     | 75.8         |
| 9           | 0.001 | 16         | 4          | 256     | 75.0         |
| ...         | ...   | ...        | ...        | ...     | ...          |
| 79          | 0.01  | 64         | 3          | 128     | 82.0         |
| 80          | 0.01  | 64         | 3          | 256     | 83.0         |
| 81          | 0.01  | 64         | 4          | 128     | 81.8         |

Now, based on the results, you can identify the best combination of hyperparameters. Let's say the best result comes from combination 80:

- **Learning rate:** 0.01

- **Batch size:** 64

- **Number of layers:** 3

- **Neurons per layer:** 256

This combination gave the highest accuracy of **83.0%**.

### Steps for Hyperparameter Tuning

- **Define the Search Space:** The first step is to define the possible values for each hyperparameter. In this case, we have 4 hyperparameters, each with 3 possible values.
- **Grid Search:** Perform a grid search to evaluate all possible combinations of the hyperparameters. This results in 81 unique combinations in our case.
- **Training and Evaluation:** For each combination, train the model using the specified hyperparameters. Evaluate the model on a validation set and record the performance metric (e.g., accuracy).
- **Selection:** Identify the combination that gives the best performance (highest accuracy) on the validation set.
- **Final Training:** Once the best hyperparameters are found, you can train the model with the best combination on the full dataset.

### Key Notes:

- **Computational Cost:** Grid Search can be computationally expensive, especially when the number of hyperparameters and their values is large. You can use **Random Search** or **Bayesian Optimization** for more efficient search strategies.
- **Cross-Validation:** Instead of a single validation split, use k-fold cross-validation to ensure the results are robust and not dependent on a particular validation set.

# Hyperparameter-Tuning\_Illustration-2

Lets assume we would like to tune the following hyperparameters for a neural network model:

**1.Learning rate** ( $lr$ ): [0.001, 0.01, 0.1]

**2.Number of hidden layers** ( $num\_layers$ ): [1, 2, 3]

**3.Number of neurons per layer** ( $neurons$ ): [64, 128]

These hyperparameters are crucial in controlling the complexity and training speed of the neural network.

Let us feed these combinations into the neural network's training process, and for each combination, record the resulting accuracy.

## Step 1: Define the Hyperparameter Grid

We have the following hyperparameters:

- $lr$  : 3 choices (0.001, 0.01, 0.1)
- $num\_layers$  : 3 choices (1, 2, 3)
- $neurons$  : 2 choices (64, 128)

This results in the following grid (combining every possible value for each hyperparameter):

$$\text{Total combinations} = 3(lr) \times 3(num\_layers) \times 2(neurons) = 18 \text{ combinations}$$

# Hyperparameter-Tuning\_Illustration-2

| Combination \ Learning Rate | 0.001 | 0.01  | 0.1   |
|-----------------------------|-------|-------|-------|
| 1 layer, 64 neurons         | 72.0% | 74.0% | 76.0% |
| 1 layer, 128 neurons        | 73.5% | 75.5% | 77.2% |
| 2 layers, 64 neurons        | 75.0% | 78.0% | 79.5% |
| 2 layers, 128 neurons       | 76.0% | 79.0% | 80.3% |
| 3 layers, 64 neurons        | 77.0% | 80.0% | 81.5% |
| 3 layers, 128 neurons       | 78.0% | 81.0% | 82.2% |

## Step 4: Identify the Best Combination

To identify the best combination of hyperparameters, we can look at the matrix and find the highest accuracy value. In this case, the highest accuracy of **82.2%** occurs with the following hyperparameter combination:

- Learning rate: 0.1
- Number of layers: 3
- Neurons per layer: 128

This combination provides the best performance based on the results in the matrix.

## Step 5: Final Training with the Best Hyperparameters

Once we identify the best combination of hyperparameters, we can retrain the neural network using `lr=0.1`, `num_layers=3`, and `neurons=128` on the full dataset (training + validation sets).



# How Hyperparameter Tuning Helps in Improving DNNs:

- Optimizing Learning Rates:**

- Learning rate** is one of the most crucial hyperparameters. If the learning rate is too high, the model may overshoot the optimal weights, leading to poor convergence. If it's too low, the model may converge too slowly, or even get stuck in local minima.

- Tuning the learning rate** helps to find the right balance, allowing the model to converge efficiently while avoiding overshooting or slow convergence. This can be achieved through strategies like learning rate schedules, cyclical learning rates, or adaptive learning rate methods (e.g., Adam, RMSProp).

- Improving Model Architecture:**

- Hyperparameters like the **number of layers, neurons per layer, activation functions, and layer types** (e.g., convolutional layers, LSTMs, etc.) define the complexity and capacity of a neural network.

- Tuning the architecture** (e.g., increasing the number of layers, changing activation functions) helps in making the model more expressive and capable of capturing complex patterns in the data. This tuning is particularly important for specific tasks (e.g., CNNs for image classification, RNNs for sequential data).

- Regularization Techniques:**

- Hyperparameters related to regularization (such as **dropout rate, L2 regularization strength**, etc.) help to control overfitting by penalizing excessively large weights or randomly dropping neurons during training.

- By tuning these **regularization hyperparameters**, you can prevent the model from memorizing the training data and improve its generalization ability, especially in cases where the model is too complex.

# How Hyperparameter Tuning Helps in Improving DNNs:

## •Batch Size and Epochs:

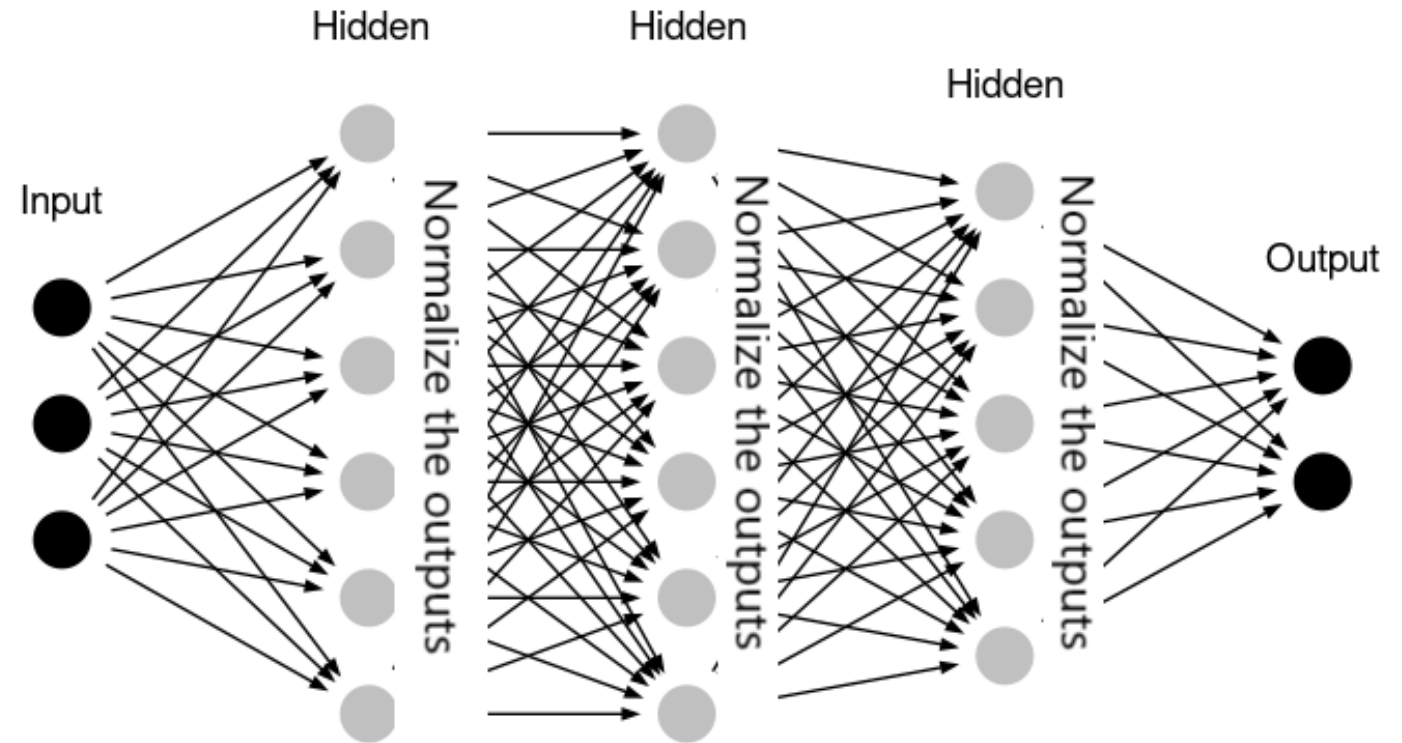
- The **batch size** determines the number of samples used to estimate the gradient during each update step, and it influences the stability and speed of training. Smaller batches generally lead to more noisy updates but can help the model escape local minima, while larger batches may provide more stable updates but take longer to converge.
- Epochs** refer to the number of times the entire training data is passed through the model. More epochs can improve the model's performance if the learning rate is well-tuned.
- Tuning batch size and epochs** helps strike a balance between computational efficiency and model performance. Using an optimal batch size can lead to faster training times without sacrificing model accuracy.

## Optimization Algorithms:

- Hyperparameters for the **optimization algorithm** (e.g., SGD, Adam, AdaGrad, RMSProp) are key in determining how efficiently the model updates its parameters during training.
- Tuning optimization algorithms, such as adjusting the **momentum**, **decay rates**, or **epsilon values**, allows the model to converge faster and avoid getting stuck in poor local minima. Some optimizers, like **Adam**, automatically adjust learning rates based on the gradient information, which can significantly speed up convergence and reduce the risk of overfitting.
- **Weight Initialization:** The method of weight initialization (e.g., Xavier, He, or random initialization) can affect how quickly the network converges and whether it reaches a good solution. Tuning weight initialization helps avoid issues like vanishing/exploding gradients and ensures the model starts with appropriate initial weights for faster and more stable convergence.

# Batch Normalization

- Batch normalization is a deep learning approach that has been shown to significantly improve the efficiency and reliability of neural network models. It is particularly useful for training very deep networks, as it can help to reduce the internal covariate shift that can occur during training.



- Batch normalization is a supervised learning method for normalizing the interlayer outputs of a neural network. As a result, the next layer receives a “reset” of the output distribution from the preceding layer, allowing it to analyze the data more effectively.

# Batch Normalization

- Batch normalization (BN) is a technique used to normalize the activations of neurons in a neural network layer, stabilizing the learning process and improving model convergence.

Steps for Batch Normalization:

1. Calculate the mean ( $\mu$ ) of the batch:

$$\mu = \frac{1}{N} \sum_{i=1}^N X_i$$

where  $N$  is the number of samples in the batch, and  $X_i$  represents each value in the batch.

2. Calculate the variance ( $\sigma^2$ ) of the batch:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (X_i - \mu)^2$$

3. Normalize the input using the formula:

$$\hat{X}_i = \frac{X_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where  $\epsilon$  is a small constant (e.g.,  $10^{-5}$ ) to avoid division by zero.

4. Apply scaling and shifting parameters (optional):

$$Y_i = \gamma \hat{X}_i + \beta$$

where  $\gamma$  (scale) and  $\beta$  (shift) are learnable parameters.

# Batch Normalization\_Solved Numerical\_1

## Problem 1: Batch Normalization for Single Feature

Let us consider the input vector  $X=[2.0,4.0,6.0,8.0]$ . Apply batch normalization on this single feature across the batch

Steps for Batch Normalization:

1. Calculate the mean ( $\mu$ ) of the batch:

$$\mu = \frac{1}{N} \sum_{i=1}^N X_i \qquad \mu = \frac{2.0 + 4.0 + 6.0 + 8.0}{4} = 5.0$$

where  $N$  is the number of samples in the batch, and  $X_i$  represents each value in the batch.

2. Calculate the variance ( $\sigma^2$ ) of the batch:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (X_i - \mu)^2$$
$$\sigma^2 = \frac{(2.0 - 5.0)^2 + (4.0 - 5.0)^2 + (6.0 - 5.0)^2 + (8.0 - 5.0)^2}{4}$$
$$\sigma^2 = \frac{9 + 1 + 1 + 9}{4} = 5.0$$

# Batch Normalization\_Solved Numerical 1

3. Normalize the input using the formula:

$$\hat{X}_i = \frac{X_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

For each value in  $X$ , this gives:

$$\hat{X}_i = \frac{X_i - 5.0}{\sqrt{5.0 + 10^{-5}}}$$

where  $\epsilon$  is a small constant (e.g.,  $10^{-5}$ ) to avoid division by zero.

4. Apply scaling and shifting parameters (optional):

$$Y_i = \gamma \hat{X}_i + \beta$$

where  $\gamma$  (scale) and  $\beta$  (shift) are learnable parameters.

(assuming  $\gamma = 1, \beta = 0$ ):

$$Y = \hat{X}$$

So, the final output is:

$$Y = [-1.3416, -0.6743, 0.6743, 1.3416]$$

$$\hat{X}_1 = \frac{2.0 - 5.0}{\sqrt{5.0 + 10^{-5}}} \approx -1.3416$$

$$\hat{X}_2 = \frac{4.0 - 5.0}{\sqrt{5.0 + 10^{-5}}} \approx -0.6743$$

$$\hat{X}_3 = \frac{6.0 - 5.0}{\sqrt{5.0 + 10^{-5}}} \approx 0.6743$$

$$\hat{X}_4 = \frac{8.0 - 5.0}{\sqrt{5.0 + 10^{-5}}} \approx 1.3416$$



# Batch Normalization\_Solved Numerical\_2

Consider a batch of data with two features:  $X = \begin{bmatrix} 2.0 & 10.0 \\ 4.0 & 20.0 \\ 6.0 & 30.0 \\ 8.0 & 40.0 \end{bmatrix}$  apply batch normalization on both features simultaneously.

Steps:

1. Calculate the mean for each feature:

$$\mu_1 = \frac{2.0 + 4.0 + 6.0 + 8.0}{4} = 5.0$$

$$\mu_2 = \frac{10.0 + 20.0 + 30.0 + 40.0}{4} = 25.0$$

2. Calculate the variance for each feature:

$$\sigma_1^2 = \frac{(2.0 - 5.0)^2 + (4.0 - 5.0)^2 + (6.0 - 5.0)^2 + (8.0 - 5.0)^2}{4} = 5.0$$

$$\sigma_2^2 = \frac{(10.0 - 25.0)^2 + (20.0 - 25.0)^2 + (30.0 - 25.0)^2 + (40.0 - 25.0)^2}{4} = 125.0$$

# Batch Normalization\_Solved Numerical\_2

Consider a batch of data with two features:  $X = \begin{bmatrix} 2.0 & 10.0 \\ 4.0 & 20.0 \\ 6.0 & 30.0 \\ 8.0 & 40.0 \end{bmatrix}$  apply batch normalization on both features simultaneously.

3. Normalize each feature using the formula:

$$\hat{X}_i = \frac{X_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

For each column in the batch, we apply normalization:

• Feature 1 normalization:

$$\hat{X}_{1,1} = \frac{2.0 - 5.0}{\sqrt{5.0 + 10^{-5}}} \approx -1.3416$$

$$\hat{X}_{1,2} = \frac{4.0 - 5.0}{\sqrt{5.0 + 10^{-5}}} \approx -0.6743$$

$$\hat{X}_{1,3} = \frac{6.0 - 5.0}{\sqrt{5.0 + 10^{-5}}} \approx 0.6743$$

$$\hat{X}_{1,4} = \frac{8.0 - 5.0}{\sqrt{5.0 + 10^{-5}}} \approx 1.3416$$

• Feature 2 normalization:

$$\hat{X}_{2,1} = \frac{10.0 - 25.0}{\sqrt{125.0 + 10^{-5}}} \approx -1.3416$$

$$\hat{X}_{2,2} = \frac{20.0 - 25.0}{\sqrt{125.0 + 10^{-5}}} \approx -0.6743$$

$$\hat{X}_{2,3} = \frac{30.0 - 25.0}{\sqrt{125.0 + 10^{-5}}} \approx 0.6743$$

$$\hat{X}_{2,4} = \frac{40.0 - 25.0}{\sqrt{125.0 + 10^{-5}}} \approx 1.3416$$

4. Apply scaling and shifting (assuming  $\gamma = 1, \beta = 0$ ):

The normalized batch after scaling and shifting is:

$$Y = \begin{bmatrix} -1.3416 & -1.3416 \\ -0.6743 & -0.6743 \\ 0.6743 & 0.6743 \\ 1.3416 & 1.3416 \end{bmatrix}$$

# Advantages of batch normalization

## Advantage:

- Stabilize the training process. Batch normalization can help to reduce the internal covariate shift that occurs during training, which can improve the stability of the training process and make it easier to optimize the model.
- Improves generalization. By normalizing the activations of a layer, batch normalization can help to reduce overfitting and improve the generalization ability of the model.
- Reduces the need for careful initialization. Batch normalization can help reduce the sensitivity of the model to the initial weights, making it easier to train the model.
- Allows for higher learning rates. Batch normalization can allow the use of higher learning rates that can speed up the training process.

## Disadvantages:

- **Dependency on batch size** can make it less effective with very small or highly variable batches.
- **Increased computational cost** and memory usage, especially for large models.
- **Poor performance with RNNs** and sequential models due to the difficulties in normalizing over time.
- **Inconsistent behavior** during inference due to differences in batch statistics used during training and testing.
- **Not beneficial for smaller networks**, where simpler solutions may be sufficient.
- **Implementation challenges in transfer learning** and fine-tuning.

# How batch normalization helps in Deep neural Networks

## 1. Reduces Internal Covariate Shift:

- **Internal covariate shift** refers to the change in the distribution of the inputs to a given layer as the parameters of the previous layers change during training. This shift can make the training process slow and unstable.
- **Batch normalization** helps to mitigate this by normalizing the activations (outputs) of each layer so that they have consistent distributions throughout training. This stabilizes the learning process, ensuring that each layer's input distribution remains relatively constant, even as the model parameters are being updated.

## 2. Speeds Up Convergence:

- Because the inputs to the layers are more stable (thanks to normalization), the network can use **higher learning rates** during training, which can accelerate convergence.
- **Batch normalization** helps the optimizer (e.g., SGD or Adam) to work more efficiently by reducing the risk of gradient vanishing/explosion. This allows the network to converge faster and reach better solutions in fewer epochs.

## 3. Acts as a Regularizer:

- Batch normalization introduces some noise into the training process because each mini-batch might have slightly different statistics (mean and variance). This noise acts similarly to **dropout**, providing a form of regularization that can help prevent overfitting.
- Because of this regularization effect, **batch normalization** can reduce the need for other regularization techniques like dropout, especially in very deep networks.

# How batch normalization helps in Deep neural Networks

## 5. Helps in Dealing with Poor Initialization:

- Deep neural networks can be sensitive to the initialization of weights, and poor initialization can lead to slow convergence or even failure to train.
- **Batch normalization** helps to alleviate this issue by ensuring that the activations are well-scaled, making the network less sensitive to the initialization of the weights.

## 6. Makes Networks Less Sensitive to Hyperparameters:

- Due to its stabilizing effect on the training process, batch normalization can make the network less sensitive to the choice of hyperparameters, such as the learning rate. This is because it reduces the fluctuations in the gradients and helps keep the training process stable even with higher learning rates.

## 7. Improves Model Generalization:

- By normalizing activations, **batch normalization** helps the model generalize better to new data. It smooths out the training process and helps the model avoid overfitting to specific details in the training data.
- It also improves the performance of the model on test and validation sets, as it helps prevent the model from fitting to noise or irrelevant patterns in the training data.

## 8. Allows for Deeper Networks:

- **Deep neural networks** tend to suffer from problems like vanishing/exploding gradients or slow training, which can limit the depth of the network. **Batch normalization** mitigates many of these issues, enabling the training of **much deeper networks** without encountering problems related to gradient flow.
- As a result, **batch normalization** enables the development of more complex models, making it easier to train very deep architectures (e.g., hundreds or even thousands of layers).

# Softmax Regression

**Softmax Regression:** Converts logits into probabilities and assigns the class with the highest probability. It's used in multiclass classification problems.

Training softmax regression involves the following steps:

1. **Initialize Parameters:**

- Initialize the weight vectors  $\mathbf{w}_i$  and biases  $b_i$  for each class.

2. **Compute the Loss:**

- For each training example, compute the logits, apply the softmax function to get the predicted probabilities, and calculate the cross-entropy loss.

3. **Backpropagation:**

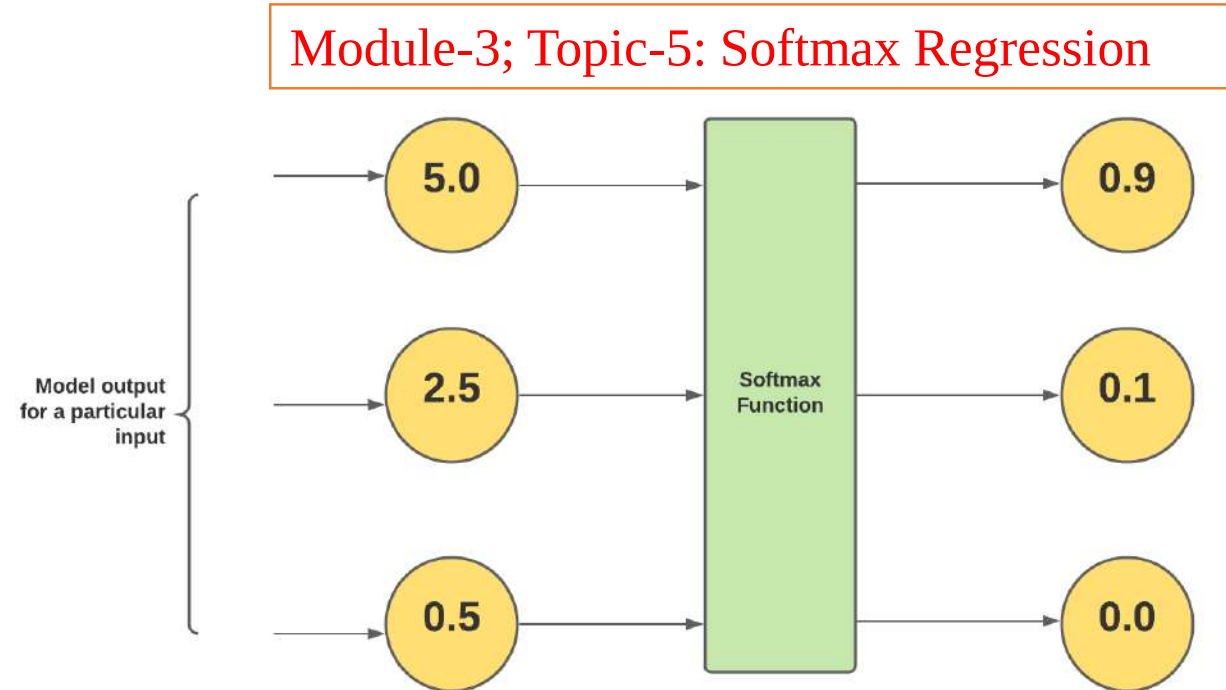
- Compute the gradient of the loss with respect to the parameters (weights and biases) using the **chain rule** of calculus. The gradients tell us how to adjust the parameters to reduce the loss.

4. **Update Parameters:**

- Update the weights and biases using an optimization algorithm like **Gradient Descent** or **Stochastic Gradient Descent (SGD)**.

5. **Repeat:**

- Repeat this process until the loss converges or a predefined number of iterations is reached.





# Softmax Regression\_Solved Numerical\_1

Assume we are dealing with a simple classification problem where we have 3 classes, and for a particular data point, the model outputs raw scores (logits) as follows:

$z=[2.0,1.0,0.1]$ . Apply softmax regression to convert these logits into probabilities.

The softmax function is defined as:

$$P(y = j|\mathbf{z}) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Where:

- $z_j$  is the score (logit) for class  $j$ ,
- $P(y = j|\mathbf{z})$  is the probability of class  $j$ ,
- The denominator sums the exponentiated scores of all classes.

1. Calculate exponentiated logits:

$$e^{z_1} = e^{2.0} \approx 7.389$$

$$e^{z_2} = e^{1.0} \approx 2.718$$

$$e^{z_3} = e^{0.1} \approx 1.105$$

2. Sum the exponentiated logits:

$$\text{sum} = 7.389 + 2.718 + 1.105 = 11.212$$

3. Compute probabilities for each class:

$$P(y = 1|\mathbf{z}) = \frac{e^{2.0}}{11.212} = \frac{7.389}{11.212} \approx 0.659$$

$$P(y = 2|\mathbf{z}) = \frac{e^{1.0}}{11.212} = \frac{2.718}{11.212} \approx 0.242$$

$$P(y = 3|\mathbf{z}) = \frac{e^{0.1}}{11.212} = \frac{1.105}{11.212} \approx 0.098$$

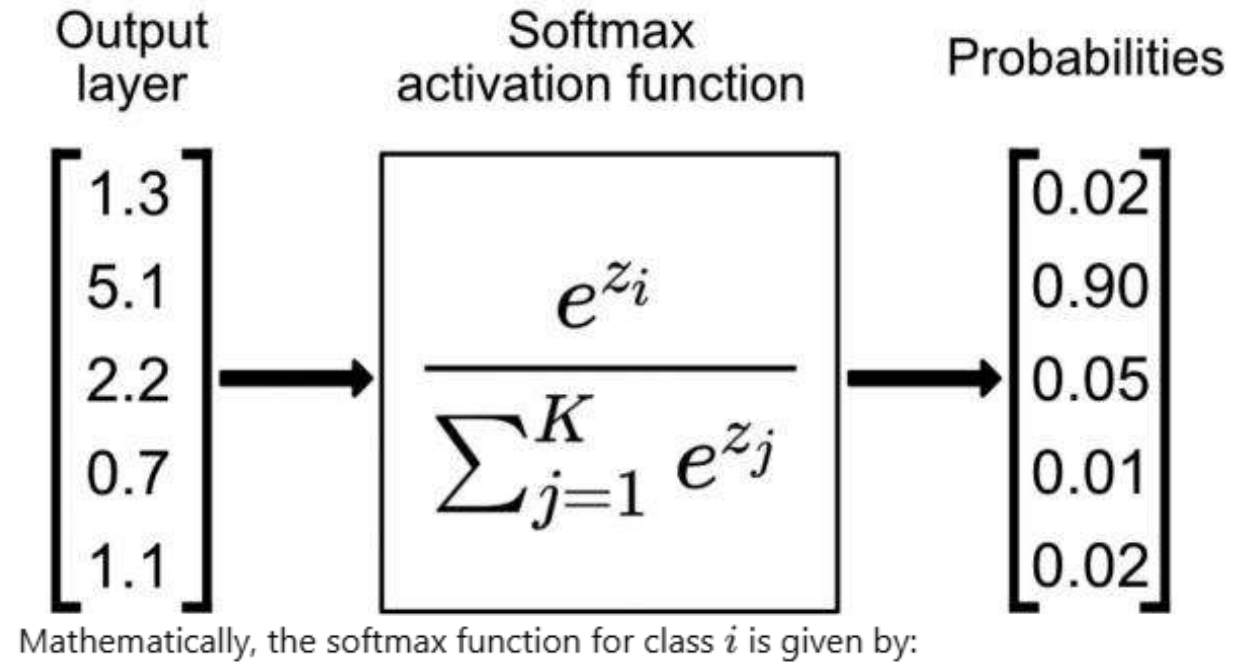
Thus, the probabilities for each class are:

- $P(y = 1|\mathbf{z}) \approx 0.659$
- $P(y = 2|\mathbf{z}) \approx 0.242$
- $P(y = 3|\mathbf{z}) \approx 0.098$

The class with the highest probability is Class 1, so **Softmax Regression** would predict Class 1.

# Softmax Classifier

- Similar to softmax regression, but generally refers to a neural network model that outputs raw scores (logits) for each class, which are then transformed into probabilities using the softmax function.
- Both models follow the same core principle: turning raw model outputs (logits) into probabilities for multiclass classification
- A Softmax classifier is commonly used in multi-class classification problems, where the goal is to assign an input to one of several possible classes.
- The softmax function converts the raw output scores (logits) of a neural network into a probability distribution, making it suitable for classification tasks. It's most often applied to the final layer of a neural network in the context of multi-class classification.



$$P(y_i|x) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Where:

- $P(y_i|x)$  is the probability of class  $i$  given the input  $x$ .
- $z_i$  is the raw score (logit) for class  $i$ .
- $C$  is the total number of classes.
- The denominator is the sum of exponentials of all the raw scores, which ensures the probabilities sum to 1.

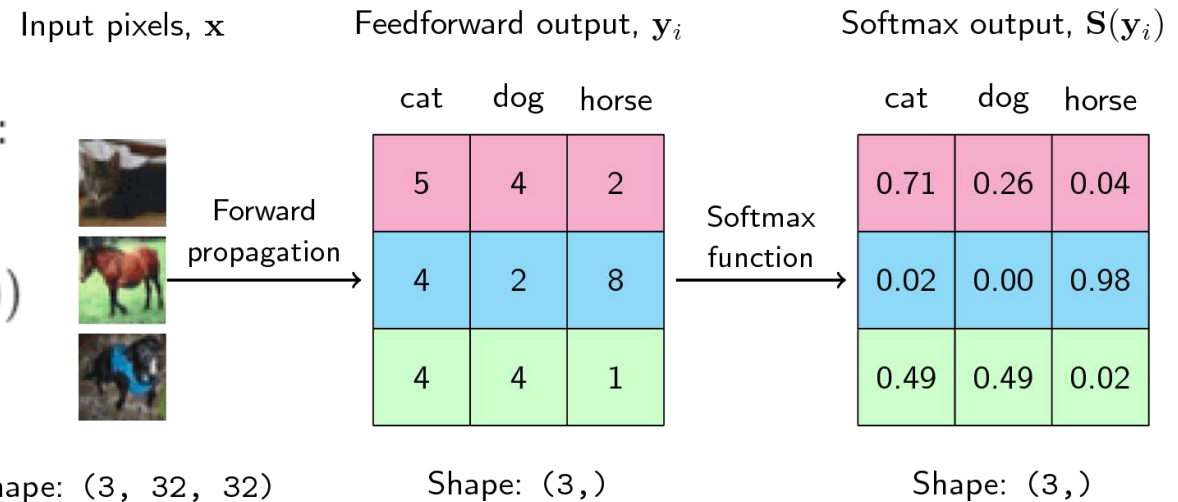
# Softmax Classifier

## Training a Softmax Classifier (Using Cross-Entropy Loss):

- A softmax classifier is typically trained using **cross-entropy loss**, which measures the dissimilarity between the predicted probability distribution (from softmax) and the actual distribution (usually represented as a one-hot encoded vector of the true class).

- The **cross-entropy loss** for a single training example is given by:

$$L = - \sum_{i=1}^C y_i \log(P(y_i|x))$$



Where:

- $y_i$  is the one-hot encoded label for class  $i$  (1 for the true class, 0 for others).
- $P(y_i|x)$  is the probability predicted by the softmax classifier for class  $i$ .
- The objective during training is to minimize the cross-entropy loss, which will result in the network learning to output probabilities that closely match the true labels.

# Softmax Classifier\_Solved Numerical\_1

Module-3; Topic-6: Softmax Classifier

- Consider a similar multiclass classification problem with a dataset where we would to classify three classes. Given an input feature vector  $\mathbf{x}=[1.0,2.0]$ , Compute the logits for each class by and apply **softmax classifier** to perform the classification.

1. Compute the logits for each class: The formula for the logits  $\mathbf{z}$  is:

$$\mathbf{z} = \mathbf{W} \cdot \mathbf{x} + \mathbf{b}$$

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.4 \\ -0.2 & 0.5 \\ 0.3 & -0.1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix}$$

Where  $\mathbf{x} = [x_1, x_2]$ ,  $\mathbf{W}$  is the weight matrix, and  $\mathbf{b}$  is the bias vector.

For class 1:

$$z_1 = (0.1 \times 1.0) + (0.4 \times 2.0) + 0.1 = 0.1 + 0.8 + 0.1 = 1.0$$

For class 2:

$$z_2 = (-0.2 \times 1.0) + (0.5 \times 2.0) - 0.1 = -0.2 + 1.0 - 0.1 = 0.7$$

For class 3:

$$z_3 = (0.3 \times 1.0) + (-0.1 \times 2.0) + 0.2 = 0.3 - 0.2 + 0.2 = 0.3$$

So the logits are:

$$\mathbf{z} = [1.0, 0.7, 0.3]$$

# Softmax Classifier\_Solved Numerical\_1

Module-3; Topic-6: Softmax Classifier

- Consider a similar multiclass classification problem with a dataset where we would to classify three classes. Given an input feature vector  $\mathbf{x}=[1.0,2.0]$ , Compute the logits for each class by and apply **softmax classifier** to perform the classification.

2. Apply the Softmax function to get probabilities:

$$P(y = j|\mathbf{z}) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.4 \\ -0.2 & 0.5 \\ 0.3 & -0.1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix}$$

First, calculate the exponentials of the logits:

$$e^{z_1} = e^{1.0} \approx 2.718$$

$$e^{z_2} = e^{0.7} \approx 2.014$$

$$e^{z_3} = e^{0.3} \approx 1.349$$

Sum the exponentiated logits:

$$\text{sum} = 2.718 + 2.014 + 1.349 = 6.081$$

Now, compute the probabilities:

$$P(y = 1|\mathbf{z}) = \frac{2.718}{6.081} \approx 0.447$$

$$P(y = 2|\mathbf{z}) = \frac{2.014}{6.081} \approx 0.331$$

$$P(y = 3|\mathbf{z}) = \frac{1.349}{6.081} \approx 0.222$$

The class with the highest probability is Class 1, so the **Softmax Classifier** would predict Class 1 for this data point.

# Softmax Classifier

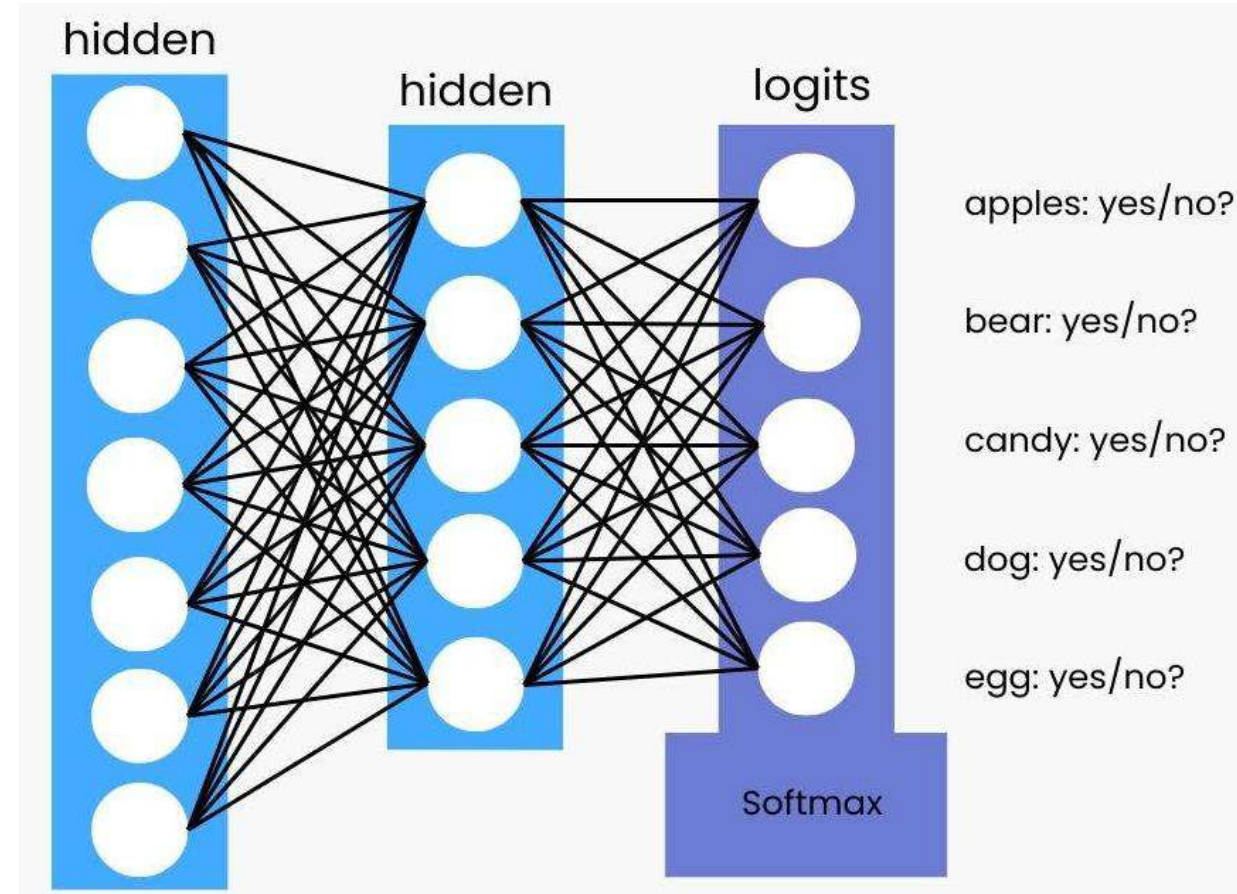
## Advantages of Softmax Classifier:

- 1.Probabilistic Output:** Softmax provides a clear, interpretable probabilistic output, making it easy to understand the confidence of the model in its predictions.
- 2.Works Well for Multi-Class Classification:** It is specifically designed for multi-class classification, where each input belongs to exactly one of many possible classes.
- 3.Differentiability:** The softmax function is differentiable, meaning it can be used in gradient-based optimization methods, which is why it's suitable for training with backpropagation.

## Disadvantages of Softmax Classifier:

- 1.Assumes Mutually Exclusive Classes:** Softmax is ideal when each class is mutually exclusive. However, it may not work well if the classes are not mutually exclusive or if there is overlap between classes.
- 2.Sensitive to Outliers:** Because softmax involves exponentiating the logits, extreme values can cause very high probabilities for certain classes, which might lead to overconfidence in predictions, even in the presence of noisy data.

## Module-3; Topic-6: Softmax Classifier



- 3. Computational Complexity:** The softmax function involves calculating the exponential of each class score, which can become computationally expensive for large numbers of classes, especially when the network is large.



# How Softmax Classifier and regression helps to improve deep neural network

- **Improved training:** By converting the raw output to a probability, softmax allows for **gradient-based optimization** to update the network's weights during training, making it possible to use backpropagation effectively.
- **Interpretable outputs:** The softmax output is easy to interpret because it provides a probability distribution, which is more meaningful for decision-making compared to raw logits.
- **Enabling multi-class classification:** Softmax is especially useful in **multi-class classification** scenarios, where there are more than two possible classes. Without softmax, the network's output would be hard to interpret, and the optimization process would be less efficient.
- **Improved Learning & Convergence:** Softmax helps to normalize outputs into probability distributions, making gradient-based optimization more stable. Regression, especially with the correct loss function, helps the network converge more effectively in continuous output tasks.
- **Task Specialization:** Both approaches allow the neural network to specialize depending on the task at hand. If the problem is classification-based (e.g., identifying an object in an image), softmax helps convert outputs into understandable classes. If it's regression-based (e.g., predicting a price), a regression approach allows for continuous value prediction.
- **Output Interpretability:** Softmax makes the model's output interpretable for classification problems (as probabilities), while regression allows the network to make useful predictions for continuous outcomes.
- **Flexibility and Versatility:** By incorporating softmax (for classification) or regression (for continuous predictions) into the deep neural network, it can be applied to a **wide variety of problems** across fields like computer vision, natural language processing, and forecasting.

# Deep Learning Frameworks

## *NumPy:*

- NumPy (Numerical Python) is a perfect tool for scientific computing and performing basic and advanced array operations.
- introduces objects for multidimensional arrays and matrices, as well as functions that allow to easily perform advanced mathematical and statistical operations on those objects
- provides vectorization of mathematical operations on arrays and matrices which significantly improves the performance
- many other python libraries are built on NumPy

**Link:** <http://www.numpy.org/>

## *SciPy:*

- Collection of algorithms for linear algebra, differential equations, numerical integration, optimization, statistics and more
- part of SciPy Stack
- built on NumPy

## *Pandas:*

- Pandas is a library created to help developers work with "labeled" and "relational" data.
- adds data structures and tools designed to work with table-like data (similar to Series and Data Frames in R)
- provides tools for data manipulation: reshaping, merging, sorting, slicing, aggregation etc.

# Deep Learning Frameworks

## *SciKit-Learn:*

- This is an industry-standard for data science projects based in Python. Scikits is a group of packages in the SciPy Stack that were created for specific functionalities – for example, image processing. Scikit-learn uses the math operations of SciPy to expose a concise interface to the most common machine learning algorithms.
- Data scientists use it for handling standard machine learning and data mining tasks such as classification, clustering, regression, model selection, model validation, dimensionality reduction, and classification.
- built on NumPy, SciPy and matplotlib

## *TensorFlow*

- TensorFlow is a popular Python framework for machine learning and deep learning, which was developed at Google Brain. It's the best tool for tasks like object identification, speech recognition, and many others. It helps in working with artificial neural networks that need to handle multiple data sets. The library includes various layer-helpers (tflearn, tf-slim, skflow), which make it even more functional. TensorFlow is constantly expanded with its new releases – including fixes in potential security vulnerabilities or improvements in the integration of TensorFlow and GPU.

## *Keras*

- Keras is a great library for building neural networks and modeling. It's very straightforward to use and provides developers with a good degree of extensibility. The library takes advantage of other packages, (Theano or TensorFlow) as its backends. Moreover, Microsoft integrated CNTK (Microsoft Cognitive Toolkit) to serve as another backend.

### *matplotlib:*

- This is a standard data science library that helps to generate data visualizations such as two-dimensional diagrams and graphs (histograms, line plots, pie charts, scatterplots, non-Cartesian coordinates graphs).
- Matplotlib is one of those plotting libraries that are really useful in data science projects — it provides an object-oriented API for embedding plots into applications.
- a set of functionalities similar to those of MATLAB
- relatively low-level; some effort needed to create advanced visualization.

### *Seaborn:*

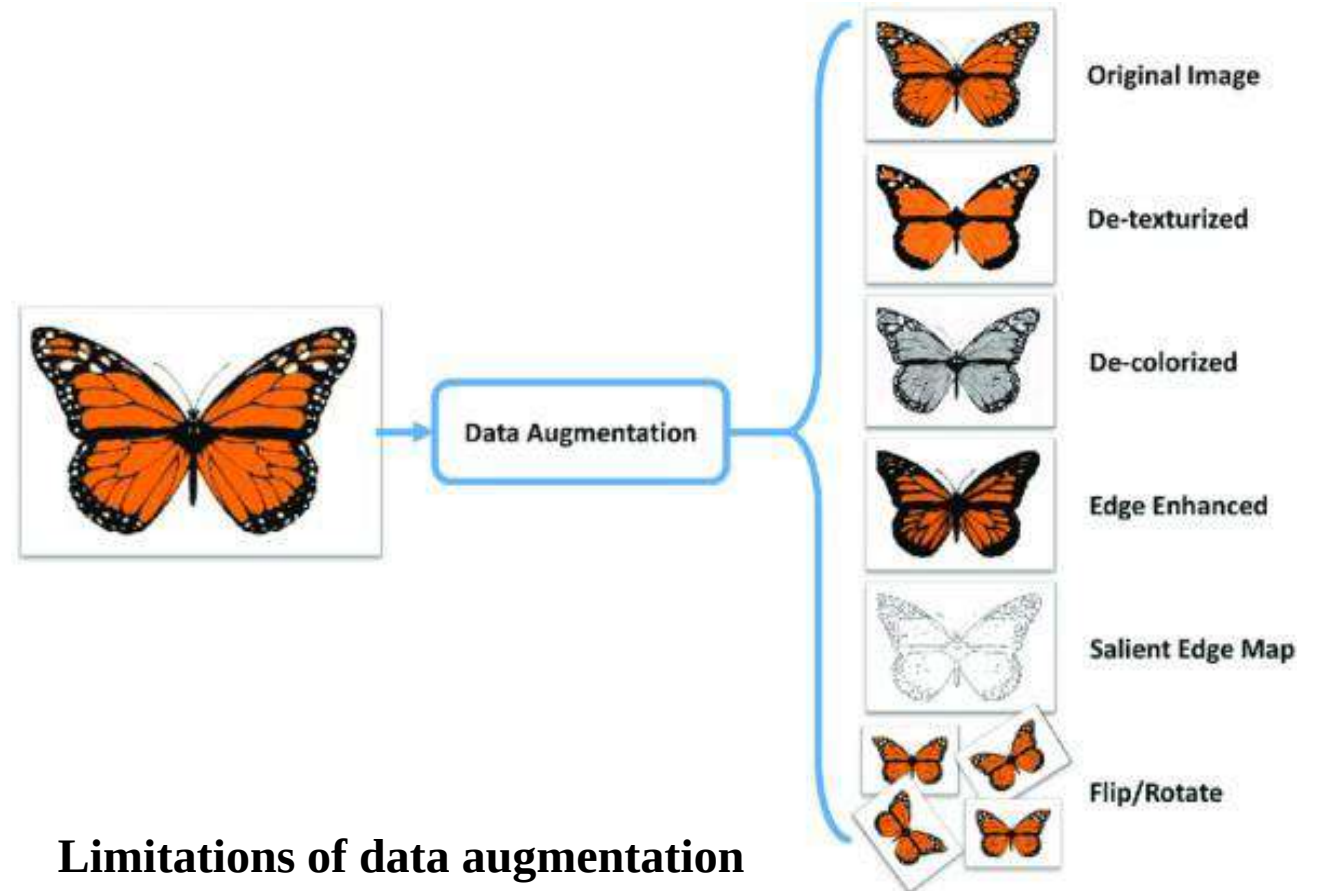
- Seaborn is based on Matplotlib and serves as a useful Python machine learning tool for visualizing statistical models – heatmaps and other types of visualizations that summarize data and depict the overall distributions. When using this library, you get to benefit from an extensive gallery of visualizations (including complex ones like time series, joint plots, and violin diagrams).
- based on matplotlib
- provides high level interface for drawing attractive statistical graphics
- Similar (in style) to the popular ggplot2 library in R

# Data Augmentation

- Data augmentation is a technique of artificially increasing the training set by creating modified copies of a dataset using existing data.
- It includes making minor changes to the dataset or using deep learning to generate new data points.
- Data augmentation helps machine learning models perform better by making the most of existing data.
- It prevents overfitting, improves accuracy, and creates diversity in training data, which is crucial when datasets are small or imbalanced.
- By simulating real-world variations, it makes models more robust and reliable—without the need for expensive data collection.

## When should you use data augmentation?

- To prevent models from overfitting.
- The initial training set is too small.
- To improve the model accuracy.
- To Reduce the operational cost of labeling and cleaning the raw dataset.



## Limitations of data augmentation

- The biases in the original dataset persist in the augmented data.
- Quality assurance for data augmentation is expensive.
- Research and development are required to build a system with advanced applications. For example, generating high-resolution images using GANs can be challenging.
- Finding an effective data augmentation approach can be challenging.

# Data Augmentation\_Numerical\_1

Module-3; Topic-7: Data Augmentation

You are given the following 3×3 matrix, representing a small image in grayscale:  
Apply the following augmentation-(1) **Rotation** (90°, 180°, 270°), (2) **Horizontal Flip**, (3) **Vertical Flip**, (4) **Zooming** (scale by 1.2x), (5) **Brightness adjustment** (randomly vary brightness)

$$\text{Original Image Matrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

(1) **Rotation** (90°, 180°, 270°)

**90° Rotation (counterclockwise):**

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

To rotate an image matrix 90° counterclockwise, we follow the rule where the first row becomes the last column, the second row becomes the second-to-last column, and so on:

$$\text{Rotated by } 90^\circ \text{ (counterclockwise)} = \begin{bmatrix} 3 & 6 & 9 \\ 2 & 5 & 8 \\ 1 & 4 & 7 \end{bmatrix}$$

**180° Rotation:**

To rotate an image matrix 180°, you can first rotate 90° counterclockwise twice (or reverse the elements row-wise and column-wise).

$$\text{Rotated by } 180^\circ = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}$$

To rotate 270° counterclockwise, it's equivalent to rotating 90° clockwise. Each row of the original image matrix becomes the first column of the rotated image:

$$\text{Rotated by } 270^\circ \text{ (counterclockwise)} = \begin{bmatrix} 7 & 4 & 1 \\ 8 & 5 & 2 \\ 9 & 6 & 3 \end{bmatrix}$$



# Data Augmentation\_Numerical\_1

You are given the following 3×3 matrix, representing a small image in grayscale:

Apply the following augmentation-(1) **Rotation** (90°, 180°, 270°), (2) **Horizontal Flip**, (3) **Vertical Flip**, (4) **Zooming** (scale by 1.2x), (5) **Brightness adjustment** (randomly vary brightness)

$$\text{Original Image Matrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

## (2) Horizontal Flip

To flip the matrix horizontally, reverse the order of elements in each row.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad \text{Horizontal Flip} = \begin{bmatrix} 3 & 2 & 1 \\ 6 & 5 & 4 \\ 9 & 8 & 7 \end{bmatrix}$$

## (3) Vertical Flip,

To flip the matrix vertically, reverse the order of rows in the matrix.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad \text{Vertical Flip} = \begin{bmatrix} 7 & 8 & 9 \\ 4 & 5 & 6 \\ 1 & 2 & 3 \end{bmatrix}$$

# Data Augmentation\_Numerical\_1

Module-3; Topic-7: Data Augmentation

You are given the following 3×3 matrix, representing a small image in grayscale:

Apply the following augmentation-(1) **Rotation** (90°, 180°, 270°), (2) **Horizontal Flip**, (3) **Vertical Flip**, (4) **Zooming** (scale by 1.2x), (5) **Brightness adjustment** (randomly vary brightness)

$$\text{Original Image Matrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

To zoom in, we need to increase the resolution of each row. We'll interpolate new values between the existing rows.

- For row 1: [1, 2, 3]
- For row 2: [4, 5, 6]
- For row 3: [7, 8, 9]

We are going to create 4 rows (instead of 3), so we will add a new row between each pair of original rows. We'll calculate the intermediate values between the original values using linear interpolation.

## First Step: Interpolating Between Rows

- Between row 1 ([1, 2, 3]) and row 2 ([4, 5, 6]):

Interpolating for each column:

$$\text{New row 1.5 (between 1st and 2nd row)} = \left[ \frac{1+4}{2}, \frac{2+5}{2}, \frac{3+6}{2} \right] = [2.5, 3.5, 4.5]$$

$$\text{New row 2.5 (between 2nd and 3rd row)} = \left[ \frac{4+7}{2}, \frac{5+8}{2}, \frac{6+9}{2} \right] = [5.5, 6.5, 7.5]$$

Thus, we have added two interpolated rows: [2.5, 3.5, 4.5] and [5.5, 6.5, 7.5].

# Data Augmentation\_Numerical\_1

Module-3; Topic-7: Data Augmentation

You are given the following 3×3 matrix, representing a small image in grayscale:

Apply the following augmentation-(1) **Rotation** (90°, 180°, 270°), (2) **Horizontal Flip**, (3) **Vertical Flip**, (4) **Zooming** (scale by 1.2x), (5) **Brightness adjustment** (randomly vary brightness)

$$\text{Original Image Matrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

## Second Step: Interpolating Between Columns

Now that we have the new rows, we need to zoom the columns. This step involves calculating the intermediate values between each adjacent column.

- **First row interpolation** between columns 1 and 2, and columns 2 and 3:

$$\text{New column values for row 1: } \left[ \frac{1+2}{2}, \frac{2+3}{2} \right] = [1.5, 2.5]$$

- Repeat for rows 2, 3, etc.

$$\text{Zoomed by 1.2x (4x4 matrix)} = \begin{bmatrix} 1 & 1.5 & 2 & 2.5 \\ 2.5 & 3 & 3.5 & 4 \\ 4.5 & 5 & 5.5 & 6 \\ 6.5 & 7 & 7.5 & 8 \end{bmatrix}$$

# Data Augmentation\_Numerical\_1

Module-3; Topic-7: Data Augmentation

You are given the following 3×3 matrix, representing a small image in grayscale:

Apply the following augmentation-(1) **Rotation** (90°, 180°, 270°), (2) **Horizontal Flip**, (3) **Vertical Flip**, (4) **Zooming** (scale by 1.2x), (5) **Brightness adjustment** (randomly vary brightness)

$$\text{Original Image Matrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

(5) **Brightness adjustment** (randomly vary brightness)

Brightness adjustment involves randomly modifying the pixel values, which can be done by multiplying each value in the matrix by a random factor. For simplicity, let's assume the random brightness adjustment factor is 1.1.

We apply the factor 1.1 to each element in the original matrix:

$$\begin{aligned} \text{Brightness Adjusted} &= \begin{bmatrix} 1 \times 1.1 & 2 \times 1.1 & 3 \times 1.1 \\ 4 \times 1.1 & 5 \times 1.1 & 6 \times 1.1 \\ 7 \times 1.1 & 8 \times 1.1 & 9 \times 1.1 \end{bmatrix} \\ &= \begin{bmatrix} 1.1 & 2.2 & 3.3 \\ 4.4 & 5.5 & 6.6 \\ 7.7 & 8.8 & 9.9 \end{bmatrix} \end{aligned}$$

# Data Augmentation\_Numerical\_2

Suppose you are training a deep learning model for image classification, and you have a dataset of 5000 images. To improve the performance of the model and prevent overfitting, you decide to apply data augmentation. You will be applying the following augmentations: (1) **Rotation** ( $90^\circ$ ,  $180^\circ$ ,  $270^\circ$ ), (2) **Horizontal Flip**, (3) **Vertical Flip**, (4) **Zooming** (scale by 1.2x), (5) **Brightness adjustment** (randomly vary brightness)

Apply each of the above augmentations independently to each image. How many unique augmented images will you have in total after applying the augmentation techniques to the original dataset?

## 1. Original Dataset:

- Number of images in the original dataset = 5000 images.

## 2. Augmentation Techniques:

- For each image, you are applying:
  - **Rotation**: 3 possible rotations ( $90^\circ$ ,  $180^\circ$ ,  $270^\circ$ ).
  - **Horizontal Flip**: 2 possible choices (flip or no flip).
  - **Vertical Flip**: 2 possible choices (flip or no flip).
  - **Zooming**: 1 possible choice (scale by 1.2x).
  - **Brightness Adjustment**: 1 possible choice (adjust brightness).

# Data Augmentation\_Numerical\_2

Suppose you are training a deep learning model for image classification, and you have a dataset of 5000 images. To improve the performance of the model and prevent overfitting, you decide to apply data augmentation. You will be applying the following augmentations: (1) **Rotation** ( $90^\circ$ ,  $180^\circ$ ,  $270^\circ$ ), (2) **Horizontal Flip**, (3) **Vertical Flip**, (4) **Zooming** (scale by 1.2x), (5) **Brightness adjustment** (randomly vary brightness)

Apply each of the above augmentations independently to each image. How many unique augmented images will you have in total after applying the augmentation techniques to the original dataset?

### 3. Calculating Total Augmented Images per Original Image:

Each augmentation is independent, so we multiply the number of possibilities for each one:

$$\text{Total Augmentations per Image} = 3 (\text{rotation}) \times 2 (\text{horizontal flip}) \times 2 (\text{vertical flip}) \times 1 (\text{zoom}) \times 1 (\text{brightness adjustment}) = 12$$

So, each image can be augmented in **12 different ways**.

### 4. Total Augmented Dataset:

Now, to get the total number of augmented images, we multiply the number of original images by the number of augmentations per image:

$$\text{Total Number of Images} = 5000 (\text{original images}) \times 12 (\text{augmented versions per image}) = 60,000$$



# Underfitting Vs Overfitting-Understanding

## Module-3; Topic-8: Underfitting Vs. Overfitting

You are training a neural network on a classification dataset and track the training loss and validation loss across multiple epochs. Below are the results for the training loss and validation loss for 3 models at different epochs.

| Epoch | Training Loss | Validation loss |
|-------|---------------|-----------------|
| 1     | 0.7           | 0.8             |
| 5     | 0.4           | 0.5             |
| 10    | 0.2           | 0.45            |
| 15    | 0.1           | 0.6             |
| 20    | 0.05          | 1.2             |

1. Epoch 1 to 5: **Underfitting**-Both Training and validation loss is high-Model is not learned well and generalize to unseen data.
2. Epoch 5 to 10: **Underfitting** to better fit-Both Training and validation loss is high-Model is decreasing.
3. Epoch 10 to 15: **Overfit**- Continuous to decrease, validation loss increasing-Model is memorising the training data and failed to generalize validation set.

### 1. Underfitting:

- Occurs when the model is too simple to learn the underlying patterns in the data.
- Both **training loss** and **validation loss** are high.
- The model fails to capture important patterns and performs poorly on both training and validation sets.

### 2. Overfitting:

- Occurs when the model is too complex and memorizes the training data.
- The **training loss** is very low, but the **validation loss** starts to increase.
- The model fits the noise in the training data, leading to poor generalization on new, unseen data.

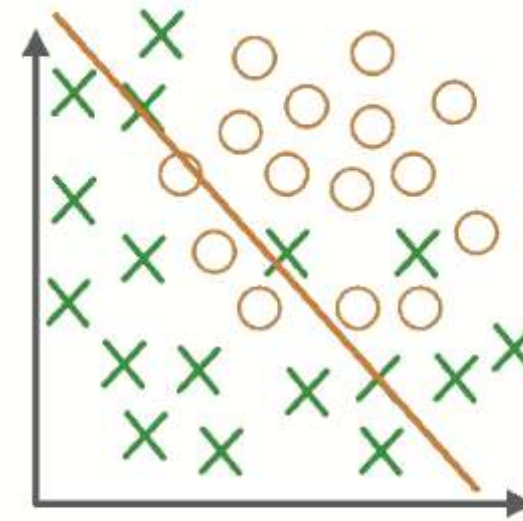
### 3. Well-Fitted (Ideal):

- Both **training loss** and **validation loss** are low and decreasing.
- The model learns well from the training data and generalizes well to unseen data.

4. Epoch 15 to 20- Overfitting Training loss low but validation loss high-Poor Generalisation.

# Overfitting and Underfitting

- **Overfitting** is a phenomenon that occurs when a model is constrained to the training set and not able to perform well on unseen data.
- That is when our model learns the noise in the training data as well. This is the case when our model memorizes the training data instead of learning the patterns in it.
- **Underfitting** on the other hand is the case when our model is not able to learn even the basic patterns available in the dataset.
- In the case of the underfitting model is unable to perform well even on the training data hence we cannot expect it to perform well on the validation data.
- This is the case when we are supposed to increase the complexity of the model or add more features to the feature set.

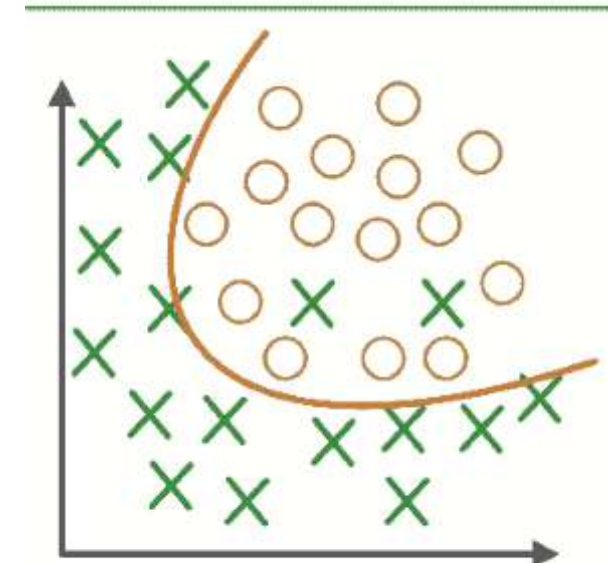


**Under-fitting**

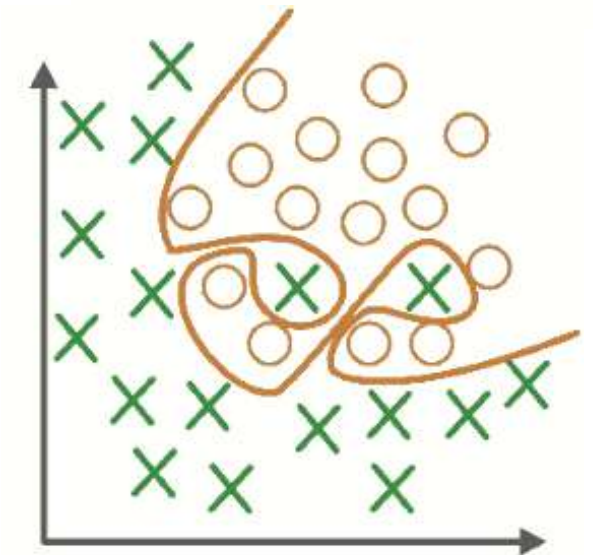
(too simple to explain the variance)

Source:

<https://www.geeksforgeeks.org/regularization-in-machine-learning/>



**Appropriate-fitting**

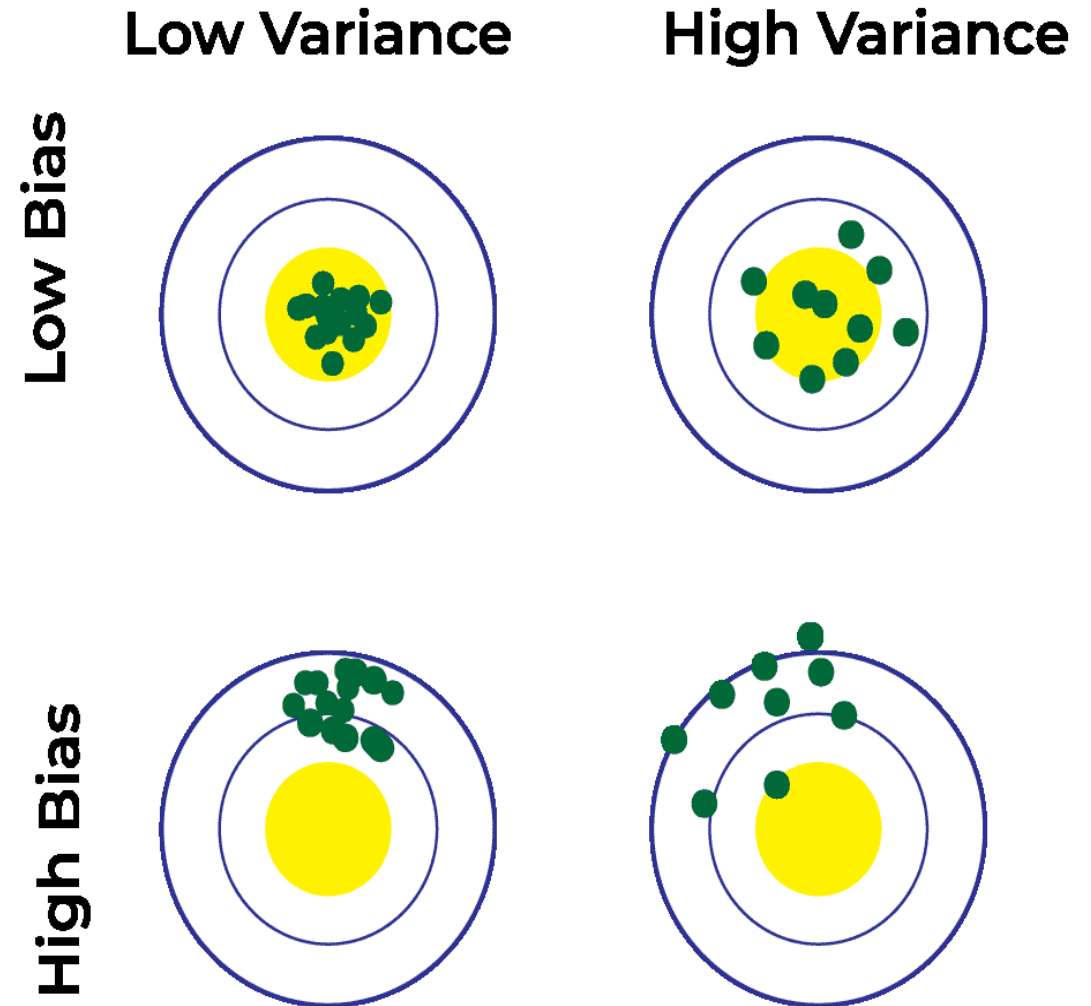


**Over-fitting**

# Regularization Functions

## Bias and Variance?

- Bias occurs when a statistical model fails to fit well on real-world data, leading to High Bias, where the model fails to learn patterns and performs poorly.
- Variance refers to the error value in predictions made using new data, while high variance occurs when the model learns from existing data noise.
- High Bias, Low Variance: A model that has high bias and low variance is considered to be underfitting.
- High Variance, Low Bias: A model that has high variance and low bias is considered to be overfitting.
- High-Bias, High-Variance: High bias and variance models struggle to capture underlying patterns and are sensitive to training data changes, resulting in unreliable and inconsistent predictions.
- Low Bias, Low Variance: A model with low bias and low variance can capture data patterns and handle variations in training data.



Source:

<https://www.geeksforgeeks.org/regularization-in-machine-learning/>

# How Overfitting Helps in Improving DNNs:

Module-3; Topic-8: Underfitting Vs. Overfitting

- **Detection and Regularization:** By observing the signs of overfitting (such as high accuracy on training data but poor performance on validation/test data), we can identify areas for improvement. Regularization techniques such as **L2 regularization (weight decay)**, **dropout**, and **early stopping** can be used to reduce overfitting, making the model more generalizable.
- **Fine-tuning the Model:** Overfitting encourages a deeper understanding of how to tune hyperparameters (e.g., learning rate, number of layers, or neurons) and select the right model complexity. By intentionally overfitting (through higher capacity models), we can gain insight into how to simplify the model for better performance.
- **Learning Curves and Validation:** When overfitting is observed, learning curves can help pinpoint when the model is starting to memorize the training data, which allows us to halt training at the optimal point or use early stopping to avoid unnecessary complexity.

## How Underfitting Helps in Improving DNNs:

- **Model Complexity Adjustment:** By recognizing underfitting, we can adjust the model's complexity (e.g., increasing the number of layers or neurons, or using a more complex architecture like a convolutional or recurrent neural network) to ensure that the model can capture enough detail from the data. Underfitting can be a signal that the model is too simplistic for the problem at hand.
- **Improved Feature Representation:** Underfitting highlights the need to reconsider the feature set and how they are represented. Sometimes the model's capacity is fine, but the data preprocessing or feature engineering needs adjustment to help the model better capture useful patterns.
- **Training Improvements:** Underfitting could also point to insufficient training. We might need to increase training time, adjust the learning rate, or improve optimization techniques to give the model a better chance to converge to a more fitting solution.

# Underfitting Vs Overfitting-Understanding

Module-3; Topic-8: Underfitting Vs. Overfitting

## Preventing Overfitting:

- **Early Stopping:** Monitor the validation loss during training and stop when it stops improving, preventing the model from learning noise.
- **Regularization (L1/L2):** Adds a penalty to the loss function to discourage large weights, making the model simpler and less likely to overfit.
- **Dropout:** Randomly deactivates neurons during training to prevent over-reliance on specific nodes.
- **Data Augmentation:** Increases the dataset size by applying random transformations (e.g., rotations or flips), helping the model generalize better.
- **Cross-Validation:** Splits the data into training and validation sets multiple times to ensure the model generalizes well across different data subsets.
- **Simplify the Model:** Reducing model complexity (e.g., fewer layers) can help avoid memorizing training data.

## Preventing Underfitting:

- **Increase Model Complexity:** Adding more layers or neurons can help the model capture complex patterns in the data.
- **Train Longer:** Allowing more epochs may give the model more time to learn the data.
- **Better Features:** Use more relevant or engineered features to improve the model's ability to capture patterns.
- **Reduce Regularization:** Too much regularization can make the model too simple, so reducing it might help.
- **Increase Training Data:** More data gives the model a better chance to learn the underlying patterns.

A balanced approach, considering the **bias-variance tradeoff**, is key to building a well-generalized model.

# Summary

- **Mini-batch Gradient Descent**- Combines full-batch and stochastic gradient descent.
  - Uses small random subsets of data (mini-batches) to update model parameters.
- **Exponential Weighted Averages**- method of averaging with weights decreasing exponentially.
  - Commonly used for smoothing or adapting to recent data.
- **Gradient Descent with Momentum**-Modifies gradient descent by adding momentum term to speed up convergence and helps in escaping from local minima.
- **RMSProp and Adam Optimization:**
  - RMSProp adjusts the learning rate for each parameter.
  - Adam combines momentum and RMSProp, optimizing faster with adaptive learning rates.
- **Hyperparameter Tuning:**
  - Process of adjusting model hyperparameters for optimal performance.
  - Includes grid search, random search, or Bayesian optimization.
- **Batch Normalization**- Normalizes inputs of each layer during training. Accelerates convergence and reduces internal covariate shift.
- **Softmax Regression & Classifier:**
  - Softmax regression converts logits into probabilities.
  - Softmax classifier applies softmax for multi-class classification.
- **Deep Learning Frameworks & Data Augmentation:**
  - Frameworks (TensorFlow, PyTorch) simplify model building.
  - Data augmentation increases training data by transformations.
- **Underfitting vs Overfitting:**
  - **Underfitting:** Model is too simple to capture patterns.
  - **Overfitting:** Model is too complex, fits noise in data.